Code Documentation

Classes, functions and the BCE model

Every class and every function in the back end, and every component in the front end.

Each entry names the file and the line range, so a class diagram box or a sequence-diagram lifeline can point at the exact code.

84 classes and 108 module functions in the back end; 21 components and 44 exported functions in the front end.

Repository CSIT314-InteriorDesignWebsite · commit 62e7353 · 12 November 2026

Contents

1 How to read this document 7

1.1 The five roles 7

1.2 Where each layer lives 7

1.3 The routers, and the paths they serve 7

2 Entity layer: backend/app/domain 9

2.1 SearchCriteria 9

2.2 Project 9

2.3 Quote 11

2.4 Counter 13

2.5 QuoteRequest 14

2.6 Company 15

2.7 Account 16

2.8 Category 17

2.9 Shortlist 18

2.10 Role 19

2.11 AccountStatus 20

2.12 ProjectStatus 20

2.13 QuoteRequestState 20

2.14 QuoteState 20

2.15 DomainError 20

2.16 EmailAlreadyRegistered 21

2.17 InvalidCredentials 21

2.18 AccountNotActive 21

2.19 InvalidRoleForSelfRegistration 22

2.20 WeakPassword 22

2.21 NotPermitted 22

2.22 AccountNotFound 22

2.23 CompanyAlreadyExists 22

2.24 CompanyNotFound 23

2.25 ProjectNotFound 23

2.26 QuoteRequestNotFound 23

2.27 QuoteNotFound 23

2.28 InvalidState 23

3 Entity access: backend/app/repositories 25

3.1 AccountRepository 25

3.2 CategoryRepository 25

3.3 CompanyRepository 26

3.4 ProjectRepository 26

3.5 QuoteRequestRepository 27
3.6 QuoteRepository 28
3.7 ShortlistRepository 29
4 Control: backend/app/services 31
4.1 AccountService 31
4.2 CategoryNotFound 31
4.3 CategoryExists 32
4.4 CategoryService 32
4.5 CompanyService 33
4.6 ImageUnavailable 33
4.7 load_image_bytes() 33
4.8 _read_local() 33
4.9 _read_remote() 34
4.10 _resolve() 34
4.11 _is_public() 34
4.12 is_animal() 35
4.13 _session() 35
4.14 available() 35
4.15 Reading 36
4.16 read() 36
4.17 object_tags() 36
4.18 engine_name() 36
4.19 SearchResult 37
4.20 ProjectSearchService 37
4.21 ProjectService 37
4.22 QuoteService 38
4.23 Bucket 39
4.24 Report 39
4.25 ReportService 40
4.26 ShortlistService 40
4.27 PaletteEntry 41
4.28 TagSuggestion 41
4.29 suggest_tags() 41
4.30 _merge() 42
4.31 _palette() 42
4.32 _stats() 42
4.33 _tags() 43
5 Control boundary: backend/app/api 44
5.1 _to_http() 44

5.2 list_accounts() 44
5.3 approve_account() 44
5.4 suspend_account() 44
5.5 register() 45
5.6 login() 45
5.7 read_current_account() 45
5.8 BudgetRange 46
5.9 CatalogueResponse 46
5.10 read_catalogue() 46
5.11 CategoryPublic 46
5.12 CategoryInput 47
5.13 CategoryPatch 47
5.14 _to_public() 47
5.15 _fail() 47
5.16 list_categories() 47
5.17 list_all_categories() 47
5.18 create_category() 48
5.19 update_category() 48
5.20 _to_http() 48
5.21 create_company() 49
5.22 read_my_company() 49
5.23 update_my_company() 49
5.24 list_companies() 49
5.25 read_company() 50
5.26 get_account_repository() 50
5.27 get_account_service() 50
5.28 get_company_repository() 50
5.29 get_company_service() 50
5.30 get_project_repository() 51
5.31 get_project_service() 51
5.32 get_project_search_service() 51
5.33 get_shortlist_repository() 51
5.34 get_shortlist_service() 51
5.35 get_quote_request_repository() 51
5.36 get_quote_repository() 51
5.37 get_quote_service() 52
5.38 get_category_repository() 52
5.39 get_category_service() 52
5.40 get_report_service() 52

5.41 get_current_account() 52
5.42 HealthResponse 53
5.43 read_health() 53
5.44 _read_photo() 53
5.45 store_upload() 54
5.46 tags_for_upload() 54
5.47 _to_http() 55
5.48 create_project() 55
5.49 list_my_projects() 55
5.50 list_company_projects() 56
5.51 search_projects() 56
5.52 read_project() 56
5.53 update_project() 57
5.54 delete_project() 57
5.55 record_project_view() 57
5.56 suggest_project_tags() 58
5.57 _fail() 58
5.58 _detail() 58
5.59 submit_request() 59
5.60 list_requests() 59
5.61 read_request() 59
5.62 counter_request() 59
5.63 cancel_request() 60
5.64 refuse_request() 60
5.65 answer_request() 60
5.66 list_quotes() 61
5.67 revise_quote() 61
5.68 accept_quote() 61
5.69 decline_quote() 62
5.70 BucketPublic 62
5.71 ReportPublic 62
5.72 read_report() 62
5.73 _forbidden() 63
5.74 read_shortlist() 63
5.75 save_project() 63
5.76 unsave_project() 64
6 Data transfer shapes: backend/app/schemas 65
7 Boundary layer: the browser 67
7.1 App 67

7.2 AuthProvider 67
7.3 ProtectedRoute 68
7.4 Home 68
7.5 Login 69
7.6 Register 70
7.7 Dashboard 71
7.8 Search 72
7.9 ProjectDetail 73
7.10 Saved 74
7.11 Designers 75
7.12 DesignerDetail 76
7.13 RequestQuote 76
7.14 Quotes 77
7.15 QuoteDetail 78
7.16 MyProjects 78
7.17 ProjectEditor 79
7.18 Profile 80
7.19 Admin 81
7.20 Platform 82
7.21 BudgetSlider 83
7.99 The API client 83
8 Sequence walks 85
8.1 A customer searches by photo 85
8.2 A customer requests a quote 85
8.3 A designer answers a request 85
8.4 An administrator approves an account 85
8.5 An administrator reads a report 85
9 Appendix: every file and what it holds 86

1 How to read this document

The platform is described in the Boundary-Control-Entity form. A class diagram draws every class below as a box. A sequence diagram draws the functions below as messages between lifelines. This document is the link between the two.

Each entry has the same parts: the name, the file and line range, the role, the purpose taken from the docstring, a table of attributes and methods, and a code excerpt. A long class shows its signatures; a short one shows the whole class.

1.1 The five roles

Role	Folder	What it does	Diagram form
`Boundary`	frontend/src/pages, components	Draws a screen, collects input, calls the API client	Lifeline on the left
`Boundary (API client)`	frontend/src/api.ts	One typed function per endpoint	Message to the server
`Control`	backend/app/api, backend/app/services	Checks permission, runs the use case, returns a shape	Lifeline in the middle
`Entity`	backend/app/domain	Holds the data and the rules that protect it	Class box
`Entity (access)`	backend/app/repositories	Finds and stores entity objects	Lifeline beside the entity
`Data transfer`	backend/app/schemas, frontend/src/types.ts	Shapes that cross the HTTP edge	Not drawn; shown as parameters

1.2 Where each layer lives

Role	Folder	Files	Classes	Functions
`Entity`	backend/app/domain	11	28	10
`Entity (access)`	backend/app/repositories	6	7	0
`Control`	backend/app/services	11	17	16
`Control (boundary)`	backend/app/api	12	8	68
`Data transfer`	backend/app/schemas	5	23	0

1.3 The routers, and the paths they serve

The back end exposes 43 HTTP operations. Each one is a control function; the table gives the verb, the path and the code position.

Verb	Path	Function	Where
GET		list_accounts()	admin.py:34
POST		approve_account()	admin.py:52
POST		suspend_account()	admin.py:68
POST		register()	auth.py:40
POST	/login	login()	auth.py:59
GET	/me	read_current_account()	auth.py:80
GET		read_catalogue()	catalogue.py:38
GET		list_categories()	categories.py:73
GET		list_all_categories()	categories.py:83
POST		create_category()	categories.py:100
PATCH		update_category()	categories.py:116
POST		create_company()	companies.py:40
GET		read_my_company()	companies.py:56
PATCH		update_my_company()	companies.py:70
GET		list_companies()	companies.py:87
GET		read_company()	companies.py:97
GET	/health	read_health()	health.py:25

POST		store_upload()	images.py:86
POST		tags_for_upload()	images.py:122
POST		create_project()	projects.py:60
GET		list_my_projects()	projects.py:76
GET		list_company_projects()	projects.py:102
GET		search_projects()	projects.py:115
GET		read_project()	projects.py:161
PATCH		update_project()	projects.py:177
DELETE		delete_project()	projects.py:197
POST		record_project_view()	projects.py:213
POST		suggest_project_tags()	projects.py:230
POST		submit_request()	quotes.py:63
GET		list_requests()	quotes.py:79
GET		read_request()	quotes.py:100
POST		counter_request()	quotes.py:115
POST		cancel_request()	quotes.py:134
POST		refuse_request()	quotes.py:153
POST		answer_request()	quotes.py:173
GET		list_quotes()	quotes.py:192
POST		revise_quote()	quotes.py:209
POST		accept_quote()	quotes.py:228
POST		decline_quote()	quotes.py:244
GET		read_report()	reports.py:46
GET		read_shortlist()	shortlist.py:31
POST		save_project()	shortlist.py:50
DELETE		unsave_project()	shortlist.py:75

2 Entity layer: backend/app/domain

These classes own the data. Each one guards its own rules in the constructor or in its methods, so a caller cannot create an object in a state the business forbids. This is the set of boxes for the class diagram.

2.1 SearchCriteria

backend/app/domain/project.py · lines 44–69

The filter a customer submits to the catalogue.

Attribute	Type	Default
keyword	str	''
category	str	''
style	str	''
budget_min	int | None	None
budget_max	int | None	None
tags	list[str]	field(default_factory=list)

Method	Purpose	Lines
def __post_init__() -> None	Refuse a budget range that cannot be answered.	58–69

lines 44–69

```
class SearchCriteria:
    """The filter a customer submits to the catalogue.

    An empty field does not filter. Each budget is optional, so the screen can
    send one end of the slider only.
    """

    keyword: str = ""
    category: str = ""
    style: str = ""
    budget_min: int | None = None
    budget_max: int | None = None
    tags: list[str] = field(default_factory=list)

    def __post_init__(self) -> None:
        """Refuse a budget range that cannot be answered."""
        if self.budget_min is not None and self.budget_min < 0:
            raise ValueError("Minimum budget must not be negative")
        if self.budget_max is not None and self.budget_max < 0:
            raise ValueError("Maximum budget must not be negative")
        if (
            self.budget_min is not None
            and self.budget_max is not None
            and self.budget_min > self.budget_max
        ):
            raise ValueError("Minimum budget must not exceed the maximum")
```

2.2 Project

backend/app/domain/project.py · lines 73–313

One entry in a designer's portfolio.

Attribute	Type	Default
owner_id	str	—
company_id	str	—
title	str	—
description	str	''
style	str	''
category	str	''

budget_min	int	0
budget_max	int	0
status	ProjectStatus	ProjectStatus.CURRENT
images	list[str]	field(default_factory=list)
tags	list[str]	field(default_factory=list)
view_count	int	0
shortlist_count	int	0
id	str	field(default_factory=lambda: str(uuid4()))
created_at	datetime	field(default_factory=lambda: datetime.now(timezone.utc))
updated_at	datetime | None	None

Method	Purpose	Lines
def __post_init__() -> None	Check the values a project cannot be created without.	93–107
def rename(title: str) -> None	Replace the project title.	109–117
def update_details() -> None	Replace the searchable details after checking them.	119–141
def mark_past() -> None	Mark the job as finished.	143–146
def mark_current() -> None	Mark the job as still running.	148–151
def add_image(url: str) -> None	Attach a photo address. A repeated address changes nothing.	153–163
def remove_image(url: str) -> None	Detach a photo address when it is present.	165–170
def replace_images(urls: list[str]) -> None	Replace the whole photo list, dropping blank and repeated entries.	172–178
def _clean_tags(tags: list[str]) -> list[str]	Strip, deduplicate and bound a tag list.	180–199
def replace_tags(tags: list[str]) -> None	Replace the whole tag list after checking it.	201–204
def add_tag(tag: str) -> None	Attach one tag. A repeated tag changes nothing.	206–209
def remove_tag(tag: str) -> None	Detach one tag when it is present.	211–217
def record_view() -> None	Count one look at the detail page.	219–221
def record_shortlist() -> None	Count one customer saving this project.	223–225
def withdraw_shortlist() -> None	Count one customer removing it. The count never falls below zero.	227–229
def is_owned_by(account_id: str) -> bool	Return ``True`` when ``account_id`` is the designer behind this project.	231–233
def matches(criteria: SearchCriteria) -> bool	Return ``True`` when this project answers ``criteria``.	235–269
def tag_overlap(wanted: list[str]) -> int	Return how many of ``wanted`` this project carries, ignoring case.	271–278
def specific_overlap(wanted: list[str]) -> int	Return how many of ``wanted`` are object tags this project carries.	280–292
def tag_score(wanted: list[str]) -> int	Return the ranking weight: objects count double, mood counts once.	294–303
def _require_valid_budget(minimum: int, maximum: int) -> None		306–310
def _touch() -> None		312–313

lines 73–313

```
# backend/app/domain/project.py — lines 73–313, signatures only
@dataclass
class Project:
    """One entry in a designer's portfolio."""
    owner_id: str
    company_id: str
    title: str
    description: str = ''
    style: str = ''
    category: str = ''
    budget_min: int = 0
    budget_max: int = 0
    status: ProjectStatus = ProjectStatus.CURRENT
    images: list[str] = field(default_factory=list)
    tags: list[str] = field(default_factory=list)
    view_count: int = 0
    shortlist_count: int = 0
    id: str = field(default_factory=lambda: str(uuid4(
    created_at: datetime = field(default_factory=lambda: datetime.n
    updated_at: datetime | None = None

    def __post_init__() -> None
        """Check the values a project cannot be created without."""
    def rename(title: str) -> None
        """Replace the project title."""
    def update_details() -> None
        """Replace the searchable details after checking them."""
    def mark_past() -> None
        """Mark the job as finished."""
    def mark_current() -> None
        """Mark the job as still running."""
    def add_image(url: str) -> None
        """Attach a photo address. A repeated address changes nothing."""
    def remove_image(url: str) -> None
        """Detach a photo address when it is present."""
    def replace_images(urls: list[str]) -> None
        """Replace the whole photo list, dropping blank and repeated entries."""
    def _clean_tags(tags: list[str]) -> list[str]
        """Strip, deduplicate and bound a tag list."""
    def replace_tags(tags: list[str]) -> None
        """Replace the whole tag list after checking it."""
    def add_tag(tag: str) -> None
        """Attach one tag. A repeated tag changes nothing."""
    def remove_tag(tag: str) -> None
        """Detach one tag when it is present."""
    def record_view() -> None
        """Count one look at the detail page."""
    def record_shortlist() -> None
        """Count one customer saving this project."""
    def withdraw_shortlist() -> None
        """Count one customer removing it. The count never falls below zero."""
    def is_owned_by(account_id: str) -> bool
        """Return ``True`` when ``account_id`` is the designer behind this project."""
    def matches(criteria: SearchCriteria) -> bool
        """Return ``True`` when this project answers ``criteria``."""
    def tag_overlap(wanted: list[str]) -> int
        """Return how many of ``wanted`` this project carries, ignoring case."""
    def specific_overlap(wanted: list[str]) -> int
        """Return how many of ``wanted`` are object tags this project carries."""
    def tag_score(wanted: list[str]) -> int
        """Return the ranking weight: objects count double, mood counts once."""
    @staticmethod
    def _require_valid_budget(minimum: int, maximum: int) -> None
    def _touch() -> None
```

2.3 Quote

backend/app/domain/quote.py · lines 38–131

One total, one validity date, one state.

Attribute	Type	Default

request_id	str	—
total_cents	int	—
valid_until	date	—
gst_included	bool	False
gst_rate_bp	int	GST_RATE_BP
note	str	''
version	int	1
state	QuoteState	QuoteState.OFFERED
created_at	datetime	field(default_factory=_now)
updated_at	datetime | None	None
id	str	field(default_factory=lambda: str(uuid4()))

Method	Purpose	Lines
def __post_init__() -> None	Check the amount, the note and the validity window.	53–74
def gst_cents() -> int	Return the tax inside or on top of the total.	77–79
def grand_total_cents() -> int	Return what the customer pays.	82–84
def is_expired(today: date | None = None) -> bool	Return whether the validity date passed.	86–88
def supersede() -> None	Retire this version because a newer one exists.	90–95
def accept() -> None	The customer takes this quote.	97–104
def decline(reason: str = '') -> None	The customer turns this quote down.	106–113
def expire() -> None	Close a quote whose validity date passed.	115–119
def validity_date(days: int, today: date | None = None) -> date	Return the date ``days`` from now, after checking the window.	122–128
def _touch() -> None		130–131

lines 38–131

```
# backend/app/domain/quote.py — lines 38–131, signatures only
@dataclass
class Quote:
    """One total, one validity date, one state."""
    request_id: str
    total_cents: int
    valid_until: date
    gst_included: bool = False
    gst_rate_bp: int = GST_RATE_BP
    note: str = ''
    version: int = 1
    state: QuoteState = QuoteState.OFFERED
    created_at: datetime = field(default_factory=_now)
    updated_at: datetime | None = None
    id: str = field(default_factory=lambda: str(uuid4(


    def __post_init__() -> None
        """Check the amount, the note and the validity window."""
    @property
    def gst_cents() -> int
        """Return the tax inside or on top of the total."""
    @property
    def grand_total_cents() -> int
        """Return what the customer pays."""
    def is_expired(today: date | None = None) -> bool
        """Return whether the validity date passed."""
    def supersede() -> None
        """Retire this version because a newer one exists."""
    def accept() -> None
        """The customer takes this quote."""
    def decline(reason: str = '') -> None
        """The customer turns this quote down."""
    def expire() -> None
        """Close a quote whose validity date passed."""
    @classmethod
    def validity_date(days: int, today: date | None = None) -> date
        """Return the date ``days`` from now, after checking the window."""
    def _touch() -> None
```

2.4 Counter

backend/app/domain/quote_request.py · lines 28–40

One target price the customer named, with a note.

Attribute	Type	Default
target_cents	int	—
note	str	—
created_at	datetime	—
id	str	field(default_factory=lambda: str(uuid4()))

Method	Purpose	Lines
def __post_init__() -> None		36–40

lines 28–40

```
class Counter:
    """One target price the customer named, with a note."""

    target_cents: int
    note: str
    created_at: datetime
    id: str = field(default_factory=lambda: str(uuid4()))

    def __post_init__(self) -> None:
        if self.target_cents <= 0:
            raise ValueError("A target price must be above zero")
        if len(self.note) > MAX_NOTE:
            raise ValueError(f"A note must be {MAX_NOTE} characters or fewer")
```

2.5 QuoteRequest

backend/app/domain/quote_request.py · lines 44–163

One customer's request to one company.

Attribute	Type	Default
customer_id	str	—
company_id	str	—
title	str	—
scope	str	—
budget_min_cents	int	0
budget_max_cents	int	0
target_cents	int | None	None
project_id	str | None	None
state	QuoteRequestState	QuoteRequestState.PENDING
counters	list[Counter]	field(default_factory=list)
closed_reason	str	''
consent_at	datetime | None	None
created_at	datetime	field(default_factory=_now)
updated_at	datetime | None	None
id	str	field(default_factory=lambda: str(uuid4()))

Method	Purpose	Lines
def __post_init__() -> None	Check what a request cannot be created without.	63–90
def is_open() -> bool	Return whether the request still accepts an answer.	93–95
def counter(target_cents: int, note: str) -> Counter	Record one target price from the customer.	97–108
def mark_answered() -> None	Move to answered after the designer sends a quote.	110–115
def refuse(reason: str) -> None	The designer declines to quote.	117–123
def cancel(reason: str) -> None	The customer stops the request.	125–131
def accept() -> None	The customer accepts the answer.	133–138
def decline(reason: str) -> None	The customer turns the answer down.	140–146
def expire() -> None	Close a request that ran out of time.	148–153
def is_overdue(now: datetime | None = None) -> bool	Return whether a pending request passed its open window.	155–160
def _touch() -> None		162–163

```
# backend/app/domain/quote_request.py — lines 44–163, signatures only
@dataclass
class QuoteRequest:
    """One customer's request to one company."""
    customer_id: str
    company_id: str
    title: str
    scope: str
    budget_min_cents: int = 0
    budget_max_cents: int = 0
    target_cents: int | None = None
    project_id: str | None = None
    state: QuoteRequestState = QuoteRequestState.PENDING
    counters: list[Counter] = field(default_factory=list)
    closed_reason: str = ''
    consent_at: datetime | None = None
    created_at: datetime = field(default_factory=_now)
    updated_at: datetime | None = None
    id: str = field(default_factory=lambda: str(uuid4(

    def __post_init__() -> None
        """Check what a request cannot be created without."""
    @property
    def is_open() -> bool
        """Return whether the request still accepts an answer."""
    def counter(target_cents: int, note: str) -> Counter
        """Record one target price from the customer."""
    def mark_answered() -> None
        """Move to answered after the designer sends a quote."""
    def refuse(reason: str) -> None
        """The designer declines to quote."""
    def cancel(reason: str) -> None
        """The customer stops the request."""
    def accept() -> None
        """The customer accepts the answer."""
    def decline(reason: str) -> None
        """The customer turns the answer down."""
    def expire() -> None
        """Close a request that ran out of time."""
    def is_overdue(now: datetime | None = None) -> bool
        """Return whether a pending request passed its open window."""
    def _touch() -> None
```

2.6 Company

backend/app/domain/company.py · lines 18–88

A designer's company profile.

Attribute	Type	Default
owner_id	str	—
name	str	—
description	str	''
contact_email	str	''
contact_phone	str	''
location	str	''
logo_url	str	''
id	str	field(default_factory=lambda: str(uuid4()))
created_at	datetime	field(default_factory=lambda: datetime.now(timezone.utc))
updated_at	datetime | None	None

Method	Purpose	Lines
def __post_init__() -> None	Normalise the values customers will read.	32–41
def rename(name: str) -> None	Replace the company name.	43–53

`def update_description(text: str) -> None`	Replace the description customers read.	55–58
`def update_contact(email: str, phone: str) -> None`	Replace the contact pair after checking the address has a domain.	60–68
`def set_location(location: str) -> None`	Replace the service area.	70–73
`def attach_logo(url: str) -> None`	Point the profile at a logo image.	75–81
`def is_owned_by(account_id: str) -> bool`	Return ``True`` when ``account_id`` is the designer behind this company.	83–85
`def _touch() -> None`		87–88

lines 18–88

```
# backend/app/domain/company.py — lines 18–88, signatures only
@dataclass
class Company:
    """A designer's company profile."""
    owner_id: str
    name: str
    description: str = ''
    contact_email: str = ''
    contact_phone: str = ''
    location: str = ''
    logo_url: str = ''
    id: str = field(default_factory=lambda: str(uuid4(
    created_at: datetime = field(default_factory=lambda: datetime.n
    updated_at: datetime | None = None

    def __post_init__() -> None
        """Normalise the values customers will read."""
    def rename(name: str) -> None
        """Replace the company name."""
    def update_description(text: str) -> None
        """Replace the description customers read."""
    def update_contact(email: str, phone: str) -> None
        """Replace the contact pair after checking the address has a domain."""
    def set_location(location: str) -> None
        """Replace the service area."""
    def attach_logo(url: str) -> None
        """Point the profile at a logo image."""
    def is_owned_by(account_id: str) -> bool
        """Return ``True`` when ``account_id`` is the designer behind this company."""
    def _touch() -> None
```

2.7 Account

backend/app/domain/account.py · lines 19–64

A single platform account.

Attribute	Type	Default
`email`	str	—
`display_name`	str	—
`password_hash`	str	—
`role`	Role	—
`status`	AccountStatus	AccountStatus.PENDING
`id`	str	field(default_factory=lambda: str(uuid4()))
`created_at`	datetime	field(default_factory=lambda: datetime.now(timezone.utc))

Method	Purpose	Lines
`def __post_init__() -> None`	Normalise the values identity depends on.	30–33
`def check_password(raw: str) -> bool`	Return ``True`` when ``raw`` is this account's password.	35–37
`def activate() -> None`	Move the account into the ACTIVE status.	39–41
`def suspend() -> None`	Move the account into the SUSPENDED status.	43–45

`def can_login() -> bool`	Return ``True`` only while the account is ACTIVE.	47–49
`def change_display_name(name: str) -> None`	Replace the display name with ``name``.	51–60
`def is_privileged() -> bool`	Return ``True`` for administrators and platform managers.	62–64

lines 19–64

```
class Account:
    """A single platform account."""

    email: str
    display_name: str
    password_hash: str
    role: Role
    status: AccountStatus = AccountStatus.PENDING
    id: str = field(default_factory=lambda: str(uuid4()))
    created_at: datetime = field(default_factory=lambda: datetime.now(timezone.utc))

    def __post_init__(self) -> None:
        """Normalise the values identity depends on."""
        self.email = self.email.strip().lower()
        self.display_name = self.display_name.strip()

    def check_password(self, raw: str) -> bool:
        """Return ``True`` when ``raw`` is this account's password."""
        return verify_password(raw, self.password_hash)

    def activate(self) -> None:
        """Move the account into the ACTIVE status."""
        self.status = AccountStatus.ACTIVE

    def suspend(self) -> None:
        """Move the account into the SUSPENDED status."""
        self.status = AccountStatus.SUSPENDED

    def can_login(self) -> bool:
        """Return ``True`` only while the account is ACTIVE."""
        return self.status is AccountStatus.ACTIVE

    def change_display_name(self, name: str) -> None:
        """Replace the display name with ``name``.

        Raises ``ValueError`` when the new name is blank; that guard is a
        programming error rather than a flow the API is expected to surface.
        """
        candidate = name.strip()
        if not candidate:
            raise ValueError("Display name must not be empty")
        self.display_name = candidate

    def is_privileged(self) -> bool:
        """Return ``True`` for administrators and platform managers."""
        return self.role.is_privileged
```

2.8 Category

backend/app/domain/category.py · lines 22–58

One room category, active or archived.

Attribute	Type	Default
`name`	str	—
`active`	bool	True
`id`	str	field(default_factory=lambda: str(uuid4()))
`created_at`	datetime	field(default_factory=_now)
`updated_at`	datetime | None	None

Method	Purpose	Lines
`def __post_init__() -> None`		31–34
`def rename(name: str) -> None`	Replace the label.	36–39
`def archive() -> None`	Hide the category from the dropdowns.	41–44
`def restore() -> None`	Show the category again.	46–49
`def _clean(name: str) -> str`		52–58

lines 22–58

```
class Category:
    """One room category, active or archived."""

    name: str
    active: bool = True
    id: str = field(default_factory=lambda: str(uuid4()))
    created_at: datetime = field(default_factory=_now)
    updated_at: datetime | None = None

    def __post_init__(self) -> None:
        self.name = self._clean(self.name)
        if self.updated_at is None:
            self.updated_at = self.created_at

    def rename(self, name: str) -> None:
        """Replace the label."""
        self.name = self._clean(name)
        self.updated_at = _now()

    def archive(self) -> None:
        """Hide the category from the dropdowns."""
        self.active = False
        self.updated_at = _now()

    def restore(self) -> None:
        """Show the category again."""
        self.active = True
        self.updated_at = _now()

    @staticmethod
    def _clean(name: str) -> str:
        candidate = name.strip()
        if not candidate:
            raise ValueError("A category name must not be empty")
        if len(candidate) > MAX_NAME:
            raise ValueError(f"A category name must be {MAX_NAME} characters or fewer")
        return candidate
```

2.9 Shortlist

backend/app/domain/shortlist.py · lines 17–68

The saved projects of one customer, oldest first.

Attribute	Type	Default
`customer_id`	str	—
`project_ids`	list[str]	field(default_factory=list)
`id`	str	field(default_factory=lambda: str(uuid4()))
`created_at`	datetime	field(default_factory=lambda: datetime.now(timezone.utc))
`updated_at`	datetime | None	None

Method	Purpose	Lines

def __post_init__() -> None	Drop blank and repeated entries, and refuse an oversized list.	26–36
def add(project_id: str) -> bool	Save a project. Returns ``False`` when it was already saved.	38–49
def remove(project_id: str) -> bool	Drop a project. Returns ``False`` when it was not saved.	51–58
def contains(project_id: str) -> bool	Return whether the project is saved.	60–62
def __len__() -> int		64–65
def _touch() -> None		67–68

lines 17–68

```
# backend/app/domain/shortlist.py — lines 17–68, signatures only
@dataclass
class Shortlist:
    """The saved projects of one customer, oldest first."""
    customer_id: str
    project_ids: list[str] = field(default_factory=list)
    id: str = field(default_factory=lambda: str(uuid4(
    created_at: datetime = field(default_factory=lambda: datetime.n
    updated_at: datetime | None = None

    def __post_init__() -> None
        """Drop blank and repeated entries, and refuse an oversized list."""
    def add(project_id: str) -> bool
        """Save a project. Returns ``False`` when it was already saved."""
    def remove(project_id: str) -> bool
        """Drop a project. Returns ``False`` when it was not saved."""
    def contains(project_id: str) -> bool
        """Return whether the project is saved."""
    def __len__() -> int
    def _touch() -> None
```

2.10 Role

backend/app/domain/enums.py · lines 8–24

Extends str, Enum.

The kind of actor an account represents.

Method	Purpose	Lines
def is_privileged() -> bool	Return ``True`` for the platform oversight roles.	17–19
def can_self_register() -> bool	Return ``True`` when a visitor may pick this role at sign up.	22–24

lines 8–24

```
class Role(str, Enum):
    """The kind of actor an account represents."""

    CUSTOMER = "CUSTOMER"
    DESIGNER = "DESIGNER"
    ADMIN = "ADMIN"
    PLATFORM_MANAGER = "PLATFORM_MANAGER"

    @property
    def is_privileged(self) -> bool:
        """Return ``True`` for the platform oversight roles."""
        return self in PRIVILEGED_ROLES

    @property
    def can_self_register(self) -> bool:
        """Return ``True`` when a visitor may pick this role at sign up."""
        return self in SELF_REGISTRATION_ROLES
```

2.11 AccountStatus

backend/app/domain/enums.py · lines 27–32

Extends str, Enum.

Where an account sits in its approval lifecycle.

lines 27–32

```
class AccountStatus(str, Enum):
    """Where an account sits in its approval lifecycle."""

    PENDING = "PENDING"
    ACTIVE = "ACTIVE"
    SUSPENDED = "SUSPENDED"
```

2.12 ProjectStatus

backend/app/domain/enums.py · lines 35–39

Extends str, Enum.

Where a project sits in its life.

lines 35–39

```
class ProjectStatus(str, Enum):
    """Where a project sits in its life."""

    CURRENT = "CURRENT"
    PAST = "PAST"
```

2.13 QuoteRequestState

backend/app/domain/enums.py · lines 42–51

Extends str, Enum.

Where a customer's quote request sits.

lines 42–51

```
class QuoteRequestState(str, Enum):
    """Where a customer's quote request sits."""

    PENDING = "PENDING"
    ANSWERED = "ANSWERED"
    ACCEPTED = "ACCEPTED"
    DECLINED = "DECLINED"
    CANCELLED = "CANCELLED"
    REFUSED = "REFUSED"
    EXPIRED = "EXPIRED"
```

2.14 QuoteState

backend/app/domain/enums.py · lines 54–61

Extends str, Enum.

Where one priced answer sits.

lines 54–61

```
class QuoteState(str, Enum):
    """Where one priced answer sits."""

    OFFERED = "OFFERED"
    ACCEPTED = "ACCEPTED"
    DECLINED = "DECLINED"
    SUPERSEDED = "SUPERSEDED"
    EXPIRED = "EXPIRED"
```

2.15 DomainError

backend/app/domain/errors.py · lines 12–19

Extends Exception.

Base class for every recoverable business rule failure.

Attribute	Type	Default
`default_message`	str	'The request breaks a business rule.'

Method	Purpose	Lines
`def __init__(message: str | None = None) -> None`		17–19

lines 12–19

```
class DomainError(Exception):
    """Base class for every recoverable business rule failure."""

    default_message: str = "The request breaks a business rule."

    def __init__(self, message: str | None = None) -> None:
        self.message: str = message or self.default_message
        super().__init__(self.message)
```

2.16 EmailAlreadyRegistered

backend/app/domain/errors.py · lines 22–25

Extends DomainError.

An account with the same email address already exists.

lines 22–25

```
class EmailAlreadyRegistered(DomainError):
    """An account with the same email address already exists."""

    default_message = "Email is already registered"
```

2.17 InvalidCredentials

backend/app/domain/errors.py · lines 28–31

Extends DomainError.

The supplied email and password do not identify an account.

lines 28–31

```
class InvalidCredentials(DomainError):
    """The supplied email and password do not identify an account."""

    default_message = "Invalid email or password"
```

2.18 AccountNotActive

backend/app/domain/errors.py · lines 34–41

Extends DomainError.

The account exists but its status forbids signing in.

Method	Purpose	Lines
`def __init__(status: AccountStatus) -> None`		39–41

lines 34–41

```
class AccountNotActive(DomainError):
    """The account exists but its status forbids signing in."""

    default_message = "Account cannot sign in"

    def __init__(self, status: AccountStatus) -> None:
        self.status: AccountStatus = status
        super().__init__(inactive_message(status))
```

2.19 InvalidRoleForSelfRegistration

backend/app/domain/errors.py · lines 53–56

Extends DomainError.

A visitor tried to register with a staff only role.

lines 53–56

```
class InvalidRoleForSelfRegistration(DomainError):
    """A visitor tried to register with a staff only role."""

    default_message = "This role cannot be chosen at registration"
```

2.20 WeakPassword

backend/app/domain/errors.py · lines 59–62

Extends DomainError.

The chosen password does not meet the minimum length.

lines 59–62

```
class WeakPassword(DomainError):
    """The chosen password does not meet the minimum length."""

    default_message = "Password must be at least 8 characters"
```

2.21 NotPermitted

backend/app/domain/errors.py · lines 65–68

Extends DomainError.

The caller may not perform this operation.

lines 65–68

```
class NotPermitted(DomainError):
    """The caller may not perform this operation."""

    default_message = "You are not allowed to do that"
```

2.22 AccountNotFound

backend/app/domain/errors.py · lines 71–74

Extends DomainError.

No account matches the supplied identifier.

lines 71–74

```
class AccountNotFound(DomainError):
    """No account matches the supplied identifier."""

    default_message = "Account not found"
```

2.23 CompanyAlreadyExists

backend/app/domain/errors.py · lines 77–80

Extends DomainError.

The designer already holds a company profile.

lines 77–80

```
class CompanyAlreadyExists(DomainError):
    """The designer already holds a company profile."""

    default_message = "This designer already has a company profile"
```

2.24 CompanyNotFound

backend/app/domain/errors.py · lines 83–86

Extends DomainError.

No company profile matches the request.

lines 83–86

```
class CompanyNotFound(DomainError):
    """No company profile matches the request."""

    default_message = "Company profile not found"
```

2.25 ProjectNotFound

backend/app/domain/errors.py · lines 89–92

Extends DomainError.

No project matches the request.

lines 89–92

```
class ProjectNotFound(DomainError):
    """No project matches the request."""

    default_message = "Project not found"
```

2.26 QuoteRequestNotFound

backend/app/domain/errors.py · lines 95–98

Extends DomainError.

No quote request matches the request.

lines 95–98

```
class QuoteRequestNotFound(DomainError):
    """No quote request matches the request."""

    default_message = "Quote request not found"
```

2.27 QuoteNotFound

backend/app/domain/errors.py · lines 101–104

Extends DomainError.

No quote matches the request.

lines 101–104

```
class QuoteNotFound(DomainError):
    """No quote matches the request."""

    default_message = "Quote not found"
```

2.28 InvalidState

backend/app/domain/errors.py · lines 107–110

Extends DomainError.

The record is in a state that does not allow this action.

lines 107–110

```
class InvalidState(DomainError):
    """The record is in a state that does not allow this action."""

    default_message = "The record is not in a state that allows this action"
```

3 Entity access: backend/app/repositories

A repository hides the store. A use case asks for 'the project with this id' and the repository answers with an entity, or with NotFound. Swapping the in-memory store for a database would change only these classes.

3.1 AccountRepository

backend/app/repositories/account_repository.py · lines 10–59

Keeps every account in process memory.

Method	Purpose	Lines
`def __init__() -> None`		18–21
`def add(account: Account) -> bool`	Store ``account`` unless its email is already taken.	23–35
`def get(account_id: str) -> Account | None`	Return the account with ``account_id``, or ``None``.	37–39
`def get_by_email(email: str) -> Account | None`	Return the account registered under ``email``, or ``None``.	41–46
`def list_all() -> list[Account]`	Return every stored account.	48–50
`def count() -> int`	Return how many accounts are stored.	52–54
`def clear() -> None`	Remove every stored account.	56–59

lines 10–59

```
# backend/app/repositories/account_repository.py — lines 10–59, signatures only
class AccountRepository:
    """Keeps every account in process memory."""
    def __init__() -> None
    def add(account: Account) -> bool
        """Store ``account`` unless its email is already taken."""
    def get(account_id: str) -> Account | None
        """Return the account with ``account_id``, or ``None``."""
    def get_by_email(email: str) -> Account | None
        """Return the account registered under ``email``, or ``None``."""
    def list_all() -> list[Account]
        """Return every stored account."""
    def count() -> int
        """Return how many accounts are stored."""
    def clear() -> None
        """Remove every stored account."""
```

3.2 CategoryRepository

backend/app/repositories/category_repository.py · lines 11–62

Keeps every category, active or archived.

Method	Purpose	Lines
`def __init__() -> None`		14–17
`def reset_defaults() -> None`	Replace the store with the ten default categories.	19–25
`def list_all() -> list[Category]`	Return every category, sorted by name.	27–29
`def list_active() -> list[Category]`	Return the categories a dropdown should show.	31–33
`def get(category_id: str) -> Category | None`	Return the category with ``category_id``, or ``None``.	35–37
`def find_by_name(name: str) -> Category | None`	Return the category whose label matches, ignoring case.	39–45
`def add(category: Category) -> bool`	Store a category unless its id or its name is already present.	47–58
`def count() -> int`	Return how many categories exist.	60–62

lines 11–62

```
# backend/app/repositories/category_repository.py — lines 11–62, signatures only
class CategoryRepository:
    """Keeps every category, active or archived."""
    def __init__() -> None
    def reset_defaults() -> None
        """Replace the store with the ten default categories."""
    def list_all() -> list[Category]
        """Return every category, sorted by name."""
    def list_active() -> list[Category]
        """Return the categories a dropdown should show."""
    def get(category_id: str) -> Category | None
        """Return the category with ``category_id``, or ``None``."""
    def find_by_name(name: str) -> Category | None
        """Return the category whose label matches, ignoring case."""
    def add(category: Category) -> bool
        """Store a category unless its id or its name is already present."""
    def count() -> int
        """Return how many categories exist."""
```

3.3 CompanyRepository

backend/app/repositories/company_repository.py · lines 10–57

Keeps every company in process memory.

Method	Purpose	Lines
`def __init__() -> None`		18–21
`def add(company: Company) -> bool`	Store ``company`` unless its designer already has one.	23–33
`def get(company_id: str) -> Company | None`	Return the company with ``company_id``, or ``None``.	35–37
`def get_by_owner(owner_id: str) -> Company | None`	Return the company owned by ``owner_id``, or ``None``.	39–44
`def list_all() -> list[Company]`	Return every stored company.	46–48
`def count() -> int`	Return how many companies are stored.	50–52
`def clear() -> None`	Remove every stored company.	54–57

lines 10–57

```
# backend/app/repositories/company_repository.py — lines 10–57, signatures only
class CompanyRepository:
    """Keeps every company in process memory."""
    def __init__() -> None
    def add(company: Company) -> bool
        """Store ``company`` unless its designer already has one."""
    def get(company_id: str) -> Company | None
        """Return the company with ``company_id``, or ``None``."""
    def get_by_owner(owner_id: str) -> Company | None
        """Return the company owned by ``owner_id``, or ``None``."""
    def list_all() -> list[Company]
        """Return every stored company."""
    def count() -> int
        """Return how many companies are stored."""
    def clear() -> None
        """Remove every stored company."""
```

3.4 ProjectRepository

backend/app/repositories/project_repository.py · lines 10–57

Keeps every project in process memory, keyed by id.

Method	Purpose	Lines
`def __init__() -> None`		16–18
`def add(project: Project) -> bool`	Store ``project`` unless its id is already present.	20–26
`def get(project_id: str) -> Project | None`	Return the project with ``project_id``, or ``None``.	28–30
`def list_by_owner(owner_id: str) -> list[Project]`	Return the projects owned by ``owner_id``.	32–34

`def list_by_company(company_id: str) -> list[Project]`	Return the projects published under ``company_id``.	36–40
`def list_all() -> list[Project]`	Return every stored project.	42–44
`def count() -> int`	Return how many projects are stored.	46–48
`def remove(project_id: str) -> bool`	Drop the project with ``project_id``. Returns whether it was present.	50–53
`def clear() -> None`	Remove every stored project.	55–57

lines 10–57

```
# backend/app/repositories/project_repository.py — lines 10–57, signatures only
class ProjectRepository:
    """Keeps every project in process memory, keyed by id."""
    def __init__() -> None
    def add(project: Project) -> bool
        """Store ``project`` unless its id is already present."""
    def get(project_id: str) -> Project | None
        """Return the project with ``project_id``, or ``None``."""
    def list_by_owner(owner_id: str) -> list[Project]
        """Return the projects owned by ``owner_id``."""
    def list_by_company(company_id: str) -> list[Project]
        """Return the projects published under ``company_id``."""
    def list_all() -> list[Project]
        """Return every stored project."""
    def count() -> int
        """Return how many projects are stored."""
    def remove(project_id: str) -> bool
        """Drop the project with ``project_id``. Returns whether it was present."""
    def clear() -> None
        """Remove every stored project."""
```

3.5 QuoteRequestRepository

backend/app/repositories/quote_repository.py · lines 11–48

Keeps every request in process memory, keyed by id.

Method	Purpose	Lines
`def __init__() -> None`		14–16
`def add(request: QuoteRequest) -> bool`	Store ``request`` unless its id is already present.	18–24
`def get(request_id: str) -> QuoteRequest | None`	Return the request with ``request_id``, or ``None``.	26–28
`def list_by_customer(customer_id: str) -> list[QuoteRequest]`	Return the requests a customer sent.	30–32
`def list_by_company(company_id: str) -> list[QuoteRequest]`	Return the requests a company received.	34–36
`def list_all() -> list[QuoteRequest]`	Return every stored request.	38–40
`def count() -> int`	Return how many requests are stored.	42–44
`def clear() -> None`	Remove every stored request.	46–48

lines 11–48

```
class QuoteRequestRepository:
    """Keeps every request in process memory, keyed by id."""

    def __init__(self) -> None:
        self._by_id: dict[str, QuoteRequest] = {}
        self._lock = threading.Lock()

    def add(self, request: QuoteRequest) -> bool:
        """Store ``request`` unless its id is already present."""
        with self._lock:
            if request.id in self._by_id:
                return False
            self._by_id[request.id] = request
            return True

    def get(self, request_id: str) -> QuoteRequest | None:
        """Return the request with ``request_id``, or ``None``."""
        return self._by_id.get(request_id)

    def list_by_customer(self, customer_id: str) -> list[QuoteRequest]:
        """Return the requests a customer sent."""
        return [entry for entry in self._by_id.values() if entry.customer_id == customer_id]

    def list_by_company(self, company_id: str) -> list[QuoteRequest]:
        """Return the requests a company received."""
        return [entry for entry in self._by_id.values() if entry.company_id == company_id]

    def list_all(self) -> list[QuoteRequest]:
        """Return every stored request."""
        return list(self._by_id.values())

    def count(self) -> int:
        """Return how many requests are stored."""
        return len(self._by_id)

    def clear(self) -> None:
        """Remove every stored request."""
        self._by_id.clear()
```

3.6 QuoteRepository

backend/app/repositories/quote_repository.py · lines 51–92

Keeps every quote version, keyed by id.

Method	Purpose	Lines
`def __init__() -> None`		54–56
`def add(quote: Quote) -> bool`	Store ``quote`` unless its id is already present.	58–64
`def get(quote_id: str) -> Quote | None`	Return the quote with ``quote_id``, or ``None``.	66–68
`def list_by_request(request_id: str) -> list[Quote]`	Return every version of a request's quote, oldest first.	70–75
`def latest_for_request(request_id: str) -> Quote | None`	Return the newest version of a request's quote.	77–80
`def list_all() -> list[Quote]`	Return every stored quote.	82–84
`def count() -> int`	Return how many quotes are stored.	86–88
`def clear() -> None`	Remove every stored quote.	90–92

lines 51–92

```
class QuoteRepository:
    """Keeps every quote version, keyed by id."""

    def __init__(self) -> None:
        self._by_id: dict[str, Quote] = {}
        self._lock = threading.Lock()

    def add(self, quote: Quote) -> bool:
        """Store ``quote`` unless its id is already present."""
        with self._lock:
            if quote.id in self._by_id:
                return False
            self._by_id[quote.id] = quote
            return True

    def get(self, quote_id: str) -> Quote | None:
        """Return the quote with ``quote_id``, or ``None``."""
        return self._by_id.get(quote_id)

    def list_by_request(self, request_id: str) -> list[Quote]:
        """Return every version of a request's quote, oldest first."""
        versions = [
            entry for entry in self._by_id.values() if entry.request_id == request_id
        ]
        return sorted(versions, key=lambda entry: entry.version)

    def latest_for_request(self, request_id: str) -> Quote | None:
        """Return the newest version of a request's quote."""
        versions = self.list_by_request(request_id)
        return versions[-1] if versions else None

    def list_all(self) -> list[Quote]:
        """Return every stored quote."""
        return list(self._by_id.values())

    def count(self) -> int:
        """Return how many quotes are stored."""
        return len(self._by_id)

    def clear(self) -> None:
        """Remove every stored quote."""
        self._by_id.clear()
```

3.7 ShortlistRepository

backend/app/repositories/shortlist_repository.py · lines 10–35

Keeps one shortlist per customer id.

Method	Purpose	Lines
`def __init__() -> None`		13–15
`def add(shortlist: Shortlist) -> bool`	Store ``shortlist`` unless that customer already has one.	17–23
`def get_by_customer(customer_id: str) -> Shortlist | None`	Return the shortlist of ``customer_id``, or ``None``.	25–27
`def count() -> int`	Return how many shortlists exist.	29–31
`def clear() -> None`	Remove every shortlist.	33–35

lines 10–35

```
class ShortlistRepository:
    """Keeps one shortlist per customer id."""

    def __init__(self) -> None:
        self._by_customer: dict[str, Shortlist] = {}
        self._lock = threading.Lock()

    def add(self, shortlist: Shortlist) -> bool:
        """Store ``shortlist`` unless that customer already has one."""
        with self._lock:
            if shortlist.customer_id in self._by_customer:
                return False
            self._by_customer[shortlist.customer_id] = shortlist
            return True

    def get_by_customer(self, customer_id: str) -> Shortlist | None:
        """Return the shortlist of ``customer_id``, or ``None``."""
        return self._by_customer.get(customer_id)

    def count(self) -> int:
        """Return how many shortlists exist."""
        return len(self._by_customer)

    def clear(self) -> None:
        """Remove every shortlist."""
        self._by_customer.clear()
```

4 Control: backend/app/services

A service is one use case. It checks permission, loads entities through the repositories, calls entity methods, and returns a result the API layer can send.

4.1 AccountService

backend/app/services/account_service.py · lines 31–141

Applies account business rules on top of the repository.

Method	Purpose	Lines
`def __init__(repository: AccountRepository) -> None`		34–35
`def register(email: str, password: str, display_name: str, role: Role) -> Account`	Create an account and store it.	37–54
`def authenticate(email: str, password: str) -> Account`	Return the account matching ``email`` and ``password``.	56–70
`def get(account_id: str) -> Account | None`	Return the account with ``account_id``, or ``None``.	72–74
`def list_accounts(actor: Account, status: AccountStatus | None = None) -> list[Account]`	Return accounts for an administrator, newest first.	76–84
`def approve(actor: Account, account_id: str) -> Account`	Activate an account so it may sign in.	86–90
`def suspend(actor: Account, account_id: str) -> Account`	Suspend an account so it cannot sign in again.	92–96
`def ensure_staff_account(email: str, password: str, display_name: str, role: Role) -> Account | None`	Create a staff account when the address is free.	98–117
`def _require_admin(actor: Account) -> None`		120–122
`def _require_target(actor: Account, account_id: str) -> Account`		124–131
`def _require_self_registration_role(role: Role) -> None`		134–136
`def _require_strong_password(password: str) -> None`		139–141

lines 31–141

```python
# backend/app/services/account_service.py — lines 31–141, signatures only
class AccountService:
    """Applies account business rules on top of the repository."""
    def __init__(repository: AccountRepository) -> None
    def register(email: str, password: str, display_name: str, role: Role) -> Account
        """Create an account and store it."""
    def authenticate(email: str, password: str) -> Account
        """Return the account matching ``email`` and ``password``."""
    def get(account_id: str) -> Account | None
        """Return the account with ``account_id``, or ``None``."""
    def list_accounts(actor: Account, status: AccountStatus | None = None) -> list[Account]
        """Return accounts for an administrator, newest first."""
    def approve(actor: Account, account_id: str) -> Account
        """Activate an account so it may sign in."""
    def suspend(actor: Account, account_id: str) -> Account
        """Suspend an account so it cannot sign in again."""
    def ensure_staff_account(email: str, password: str, display_name: str, role: Role) -> Account | None
        """Create a staff account when the address is free."""
    @staticmethod
    def _require_admin(actor: Account) -> None
    def _require_target(actor: Account, account_id: str) -> Account
    @staticmethod
    def _require_self_registration_role(role: Role) -> None
    @staticmethod
    def _require_strong_password(password: str) -> None
```

4.2 CategoryNotFound

backend/app/services/category_service.py · lines 12–16

No category matches the supplied id.

Method	Purpose	Lines
`def __init__() -> None`		15–16

lines 12–16

```
class CategoryNotFound(Exception):
    """No category matches the supplied id."""

    def __init__(self) -> None:
        super().__init__("Category not found")
```

4.3 CategoryExists

backend/app/services/category_service.py · lines 19–23

A category with that label already exists.

Method	Purpose	Lines
`def __init__() -> None`		22–23

lines 19–23

```
class CategoryExists(Exception):
    """A category with that label already exists."""

    def __init__(self) -> None:
        super().__init__("A category with that name already exists")
```

4.4 CategoryService

backend/app/services/category_service.py · lines 26–87

Applies the staff rules on top of the category store.

Method	Purpose	Lines
`def __init__(categories: CategoryRepository) -> None`		29–30
`def list_active() -> list[Category]`	Return the labels a dropdown should show.	32–34
`def list_all(actor: Account) -> list[Category]`	Return every category, archived ones included.	36–39
`def add(actor: Account, name: str) -> Category`	Create a category.	41–50
`def rename(actor: Account, category_id: str, name: str) -> Category`	Change a category label.	52–62
`def get(actor: Account, category_id: str) -> Category`	Return one category for staff.	64–70
`def set_active(actor: Account, category_id: str, active: bool) -> Category`	Archive or restore a category.	72–82
`def _require_staff(actor: Account) -> None`		85–87

lines 26–87

```
# backend/app/services/category_service.py — lines 26–87, signatures only
class CategoryService:
    """Applies the staff rules on top of the category store."""
    def __init__(categories: CategoryRepository) -> None
    def list_active() -> list[Category]
        """Return the labels a dropdown should show."""
    def list_all(actor: Account) -> list[Category]
        """Return every category, archived ones included."""
    def add(actor: Account, name: str) -> Category
        """Create a category."""
    def rename(actor: Account, category_id: str, name: str) -> Category
        """Change a category label."""
    def get(actor: Account, category_id: str) -> Category
        """Return one category for staff."""
    def set_active(actor: Account, category_id: str, active: bool) -> Category
        """Archive or restore a category."""
    @staticmethod
    def _require_staff(actor: Account) -> None
```

4.5 CompanyService

backend/app/services/company_service.py · lines 12–100

Applies the company profile rules on top of the repository.

Method	Purpose	Lines
`def __init__(companies: CompanyRepository) -> None`		15–16
`def create(actor: Account, name: str, description: str, contact_email: str, contact_phone: str, location: str, logo_url: str = '') -> Company`	Create the company profile for ``actor``.	18–44
`def get_for_owner(actor: Account) -> Company`	Return the profile owned by ``actor``.	46–52
`def update(actor: Account) -> Company`	Change only the fields that were supplied.	54–84
`def get(company_id: str) -> Company`	Return a company by id. Any signed in account may read it.	86–91
`def list_companies() -> list[Company]`	Return every company, sorted by name for a stable listing.	93–95
`def _require_designer(actor: Account) -> None`		98–100

lines 12–100

```
# backend/app/services/company_service.py — lines 12–100, signatures only
class CompanyService:
    """Applies the company profile rules on top of the repository."""
    def __init__(companies: CompanyRepository) -> None
    def create(actor: Account, name: str, description: str, contact_email: str, contact_phone: str, location: str,
logo_url: str = '') -> Company
        """Create the company profile for ``actor``."""
    def get_for_owner(actor: Account) -> Company
        """Return the profile owned by ``actor``."""
    def update(actor: Account) -> Company
        """Change only the fields that were supplied."""
    def get(company_id: str) -> Company
        """Return a company by id. Any signed in account may read it."""
    def list_companies() -> list[Company]
        """Return every company, sorted by name for a stable listing."""
    @staticmethod
    def _require_designer(actor: Account) -> None
```

4.6 ImageUnavailable

backend/app/services/image_source.py · lines 28–29

The photo cannot be read, or it must not be read.

lines 28–29

```
class ImageUnavailable(Exception):
    """The photo cannot be read, or it must not be read."""
```

4.7 load_image_bytes()

backend/app/services/image_source.py · lines 32–39

Return the bytes of the photo that ``reference`` points at.

lines 32–39

```
def load_image_bytes(reference: str, public_dir: Path) -> bytes:
    """Return the bytes of the photo that ``reference`` points at."""
    candidate = reference.strip()
    if not candidate:
        raise ImageUnavailable("The project has no photo to read")
    if candidate.startswith("/"):
        return _read_local(candidate, public_dir)
    return _read_remote(candidate)
```

4.8 _read_local()

backend/app/services/image_source.py · lines 42–51

lines 42–51

```
def _read_local(reference: str, public_dir: Path) -> bytes:
    root = public_dir.resolve()
    target = (root / reference.lstrip("/")).resolve()
    if target != root and root not in target.parents:
        raise ImageUnavailable("The photo path leaves the public folder")
    if not target.is_file():
        raise ImageUnavailable("The photo is not in the public folder")
    if target.stat().st_size > MAX_BYTES:
        raise ImageUnavailable("The photo is larger than 5 MB")
    return target.read_bytes()
```

4.9 _read_remote()

backend/app/services/image_source.py · lines 54–82

lines 54–82

```
def _read_remote(reference: str) -> bytes:
    parsed = urlparse(reference)
    if parsed.scheme not in ("http", "https") or not parsed.hostname:
        raise ImageUnavailable("Only http and https photo addresses are accepted")
    for address in _resolve(parsed.hostname):
        if not _is_public(address):
            raise ImageUnavailable("The photo host is not a public address")

    chunks: list[bytes] = []
    size = 0
    try:
        with httpx.stream(
            "GET", reference, timeout=TIMEOUT_SECONDS, follow_redirects=False
        ) as response:
            if response.status_code != 200:
                raise ImageUnavailable("The photo could not be downloaded")
            content_type = (
                response.headers.get("content-type", "").split(";")[0].strip().lower()
            )
            if content_type not in ALLOWED_TYPES:
                raise ImageUnavailable("The address does not hold an image")
            for chunk in response.iter_bytes():
                size += len(chunk)
                if size > MAX_BYTES:
                    raise ImageUnavailable("The photo is larger than 5 MB")
                chunks.append(chunk)
    except httpx.HTTPError as exc:
        raise ImageUnavailable("The photo could not be downloaded") from exc
    return b"".join(chunks)
```

4.10 _resolve()

backend/app/services/image_source.py · lines 85–90

lines 85–90

```
def _resolve(host: str) -> list[str]:
    try:
        infos = socket.getaddrinfo(host, None)
    except socket.gaierror as exc:
        raise ImageUnavailable("The photo host does not resolve") from exc
    return sorted({info[4][0] for info in infos})
```

4.11 _is_public()

backend/app/services/image_source.py · lines 93–105

lines 93–105

```
def _is_public(address: str) -> bool:
    try:
        parsed = ipaddress.ip_address(address)
    except ValueError:
        return False
    return not (
        parsed.is_private
        or parsed.is_loopback
        or parsed.is_link_local
        or parsed.is_reserved
        or parsed.is_multicast
        or parsed.is_unspecified
    )
```

4.12 is_animal()

backend/app/services/object_labels.py · lines 122–129

Return True when the label names an animal.

lines 122–129

```
def is_animal(label: str) -> bool:
    """Return True when the label names an animal.


    Only the first synonym is matched, so "potted plant" cannot trip on "ant",
    and the word boundaries keep "car" away from "cardoon".
    """
    primary = label.split(",")[0].strip()
    return bool(ANIMAL_PATTERN.search(primary))
```

4.13 _session()

backend/app/services/object_labels.py · lines 133–153

@lru_cache(maxsize=1)

Load the model once. Return None when the model or runtime is absent.

lines 133–153

```
def _session():
    """Load the model once. Return None when the model or runtime is absent."""
    if not MODEL_PATH.is_file():
        return None
    labels_file = MODEL_PATH.with_name("synset.txt")
    if not labels_file.is_file():
        return None
    try:
        import onnxruntime as ort
        from PIL import Image  # noqa: F401  - proves Pillow is present
    except ImportError:
        return None
    session = ort.InferenceSession(
        str(MODEL_PATH), providers=["CPUExecutionProvider"]
    )
    labels = [
        line.split(" ", 1)[1] if " " in line else line
        for line in labels_file.read_text(encoding="utf-8").splitlines()
        if line.strip()
    ]
    return session, labels
```

4.14 available()

backend/app/services/object_labels.py · lines 156–158

Return True when a model can answer.

lines 156–158

```
def available() -> bool:
    """Return True when a model can answer."""
    return _session() is not None
```

4.15 Reading

backend/app/services/object_labels.py · lines 162–169

What one inference pass saw. The diagnostics drive the interior gate.

lines 162–169

```
class Reading:
    """What one inference pass saw. The diagnostics drive the interior gate."""


    tags: list[str]
    top_label: str
    top_probability: float
    interior_probability: float
    subject: str
```

4.16 read()

backend/app/services/object_labels.py · lines 172–231

Run one inference pass and return the tags plus the diagnostics.

lines 172–231

```
# backend/app/services/object_labels.py — lines 172–231, first 20 lines of 60
def read(data: bytes) -> Reading | None:
    """Run one inference pass and return the tags plus the diagnostics.


    ``None`` means the model is absent or the bytes are not readable. Callers
    must fall back to the colour stage, not fail.
    ``interior_probability`` is the best probability among the classes that map
    to a catalogue tag. A photo of a monkey scores near zero there even when the
    model is confident about the monkey.
    """
    loaded = _session()
    if loaded is None:
        return None
    session, labels = loaded
    try:
        import numpy as np
        from PIL import Image


        with Image.open(BytesIO(data)) as image:
            image.load()
            small = image.convert("RGB").resize(SIZE)
    ...
```

4.17 object_tags()

backend/app/services/object_labels.py · lines 234–241

Return up to ``LIMIT`` catalogue tags for ``data``, most confident first.

lines 234–241

```
def object_tags(data: bytes) -> list[str]:
    """Return up to ``LIMIT`` catalogue tags for ``data``, most confident first.


    An empty list means the model is absent, refused the bytes, or saw nothing
    it can name. The caller must treat that as "colour tags only".
    """
    reading = read(data)
    return [] if reading is None else reading.tags
```

4.18 engine_name()

backend/app/services/object_labels.py · lines 244–246

Name the analyser for the response, so the screen can be honest.

lines 244–246

```
def engine_name() -> str:
    """Name the analyser for the response, so the screen can be honest."""
    return "objects and colour" if available() else "colour only"
```

4.19 SearchResult

backend/app/services/project_search_service.py · lines 20–33

One page of matches plus the numbers the screen needs.

Method	Purpose	Lines
`def pages() -> int`	Return how many pages the total fills, at least one.	29–33

lines 20–33

```
class SearchResult:
    """One page of matches plus the numbers the screen needs."""

    items: list[Project]
    total: int
    page: int
    page_size: int

    @property
    def pages(self) -> int:
        """Return how many pages the total fills, at least one."""
        if self.page_size <= 0:
            return 1
        return max(1, -(-self.total // self.page_size))
```

4.20 ProjectSearchService

backend/app/services/project_search_service.py · lines 36–92

Filters the catalogue. Holds no state of its own.

Method	Purpose	Lines
`def __init__(projects: ProjectRepository) -> None`		39–40
`def search() -> SearchResult`	Return the matches for one page of results.	42–92

lines 36–92

```
# backend/app/services/project_search_service.py — lines 36–92, signatures only
class ProjectSearchService:
    """Filters the catalogue. Holds no state of its own."""
    def __init__(projects: ProjectRepository) -> None
    def search() -> SearchResult
        """Return the matches for one page of results."""
```

4.21 ProjectService

backend/app/services/project_service.py · lines 13–178

Applies the portfolio rules on top of the two stores.

Method	Purpose	Lines
`def __init__(projects: ProjectRepository, companies: CompanyRepository) -> None`		16–22
`def create(actor: Account) -> Project`	Publish a project under the caller's company.	24–61
`def get(project_id: str) -> Project`	Return a project by id. Any signed in account may read it.	63–68
`def update(actor: Account, project_id: str) -> Project`	Change only the fields that were supplied. Owner only.	70–108
`def require_owned(actor: Account, project_id: str) -> Project`	Return the project only when the caller owns it.	110–112
`def delete(actor: Account, project_id: str) -> None`	Remove a project. Owner only.	114–117
`def list_for_owner(actor: Account, status: ProjectStatus | None = None, name: str = '') -> list[Project]`	List the owner's projects, optionally filtered by status and title.	119–156
`def list_for_company(company_id: str) -> list[Project]`	Return the projects published under one	158–164

	company, newest first.	
def _require_designer(actor: Account) -> None		167–169
def _require_owned(actor: Account, project_id: str) -> Project		171–178

lines 13–178

```
# backend/app/services/project_service.py — lines 13–178, signatures only
class ProjectService:
    """Applies the portfolio rules on top of the two stores."""
    def __init__(projects: ProjectRepository, companies: CompanyRepository) -> None
    def create(actor: Account) -> Project
        """Publish a project under the caller's company."""
    def get(project_id: str) -> Project
        """Return a project by id. Any signed in account may read it."""
    def update(actor: Account, project_id: str) -> Project
        """Change only the fields that were supplied. Owner only."""
    def require_owned(actor: Account, project_id: str) -> Project
        """Return the project only when the caller owns it."""
    def delete(actor: Account, project_id: str) -> None
        """Remove a project. Owner only."""
    def list_for_owner(actor: Account, status: ProjectStatus | None = None, name: str = '') -> list[Project]
        """List the owner's projects, optionally filtered by status and title."""
    def list_for_company(company_id: str) -> list[Project]
        """Return the projects published under one company, newest first."""
    @staticmethod
    def _require_designer(actor: Account) -> None
    def _require_owned(actor: Account, project_id: str) -> Project
```

4.22 QuoteService

backend/app/services/quote_service.py · lines 32–312

Applies the quoting rules on top of the four stores.

Method	Purpose	Lines
def __init__(requests: QuoteRequestRepository, quotes: QuoteRepository, companies: CompanyRepository, projects: ProjectRepository) -> None		35–45
def submit(actor: Account) -> QuoteRequest	Send one request to one company.	49–88
def counter(actor: Account, request_id: str, target_cents: int, note: str) -> QuoteRequest	Record the price the customer wants and why.	90–99
def cancel(actor: Account, request_id: str, reason: str = '') -> QuoteRequest	Stop an open request.	101–110
def accept(actor: Account, quote_id: str) -> Quote	Accept one quote. Other requests of this customer stay open.	112–124
def decline(actor: Account, quote_id: str, reason: str = '') -> Quote	Turn one quote down.	126–134
def answer(actor: Account, request_id: str) -> Quote	Send version one of the quote.	138–165
def revise(actor: Account, quote_id: str) -> Quote	Send the next version of the quote.	167–202
def refuse(actor: Account, request_id: str, reason: str = '') -> QuoteRequest	Decline to quote.	204–211
def list_for_customer(actor: Account, state: QuoteRequestState | None = None) -> list[QuoteRequest]	Return the caller's requests, newest first.	215–220
def list_for_designer(actor: Account, state: QuoteRequestState | None = None) -> list[QuoteRequest]	Return the requests sent to the caller's company, newest first.	222–227
def get_request(actor: Account, request_id: str) -> QuoteRequest	Return one request to a party that may see it.	229–237
def quotes_for(actor: Account, request_id: str) -> list[Quote]	Return every version of a request's quote.	239–242
def _sorted(requests: list[QuoteRequest], state: QuoteRequestState | None) -> list[QuoteRequest]		246–253
def _refresh(request: QuoteRequest) -> None	Close what time has closed. Called on every read.	255–264
def _may_read(actor: Account, request: QuoteRequest) -> bool		266–272
def _owned_request(actor: Account, request_id: str) -> QuoteRequest		274–281

def _owned_quote(actor: Account, quote_id: str) -> tuple[Quote, QuoteRequest]		283–288
def _request_for_designer(actor: Account, request_id: str) -> QuoteRequest		290–299
def _company_of(actor: Account)		301–307
def _require_customer(actor: Account) -> None		310–312

lines 32–312

```
# backend/app/services/quote_service.py — lines 32–312, signatures only
class QuoteService:
    """Applies the quoting rules on top of the four stores."""
    def __init__(requests: QuoteRequestRepository, quotes: QuoteRepository, companies: CompanyRepository, projects:
ProjectRepository) -> None
    def submit(actor: Account) -> QuoteRequest
        """Send one request to one company."""
    def counter(actor: Account, request_id: str, target_cents: int, note: str) -> QuoteRequest
        """Record the price the customer wants and why."""
    def cancel(actor: Account, request_id: str, reason: str = '') -> QuoteRequest
        """Stop an open request."""
    def accept(actor: Account, quote_id: str) -> Quote
        """Accept one quote. Other requests of this customer stay open."""
    def decline(actor: Account, quote_id: str, reason: str = '') -> Quote
        """Turn one quote down."""
    def answer(actor: Account, request_id: str) -> Quote
        """Send version one of the quote."""
    def revise(actor: Account, quote_id: str) -> Quote
        """Send the next version of the quote."""
    def refuse(actor: Account, request_id: str, reason: str = '') -> QuoteRequest
        """Decline to quote."""
    def list_for_customer(actor: Account, state: QuoteRequestState | None = None) -> list[QuoteRequest]
        """Return the caller's requests, newest first."""
    def list_for_designer(actor: Account, state: QuoteRequestState | None = None) -> list[QuoteRequest]
        """Return the requests sent to the caller's company, newest first."""
    def get_request(actor: Account, request_id: str) -> QuoteRequest
        """Return one request to a party that may see it."""
    def quotes_for(actor: Account, request_id: str) -> list[Quote]
        """Return every version of a request's quote."""
    def _sorted(requests: list[QuoteRequest], state: QuoteRequestState | None) -> list[QuoteRequest]
    def _refresh(request: QuoteRequest) -> None
        """Close what time has closed. Called on every read."""
    def _may_read(actor: Account, request: QuoteRequest) -> bool
    def _owned_request(actor: Account, request_id: str) -> QuoteRequest
    def _owned_quote(actor: Account, quote_id: str) -> tuple[Quote, QuoteRequest]
    def _request_for_designer(actor: Account, request_id: str) -> QuoteRequest
    def _company_of(actor: Account)
    @staticmethod
    def _require_customer(actor: Account) -> None
```

4.23 Bucket

backend/app/services/report_service.py · lines 22–31

One period of activity.

lines 22–31

```
class Bucket:
    """One period of activity."""


    label: str
    start: datetime
    end: datetime
    accounts: int
    projects: int
    quote_requests: int
    quotes: int
```

4.24 Report

backend/app/services/report_service.py · lines 35–39

A period choice and the buckets it produced.

lines 35–39

```
class Report:
    """A period choice and the buckets it produced."""


    period: str
    buckets: list[Bucket]
```

4.25 ReportService

backend/app/services/report_service.py · lines 42–133

Builds reports from the four stores that hold timestamps.

Method	Purpose	Lines
`def __init__(accounts: AccountRepository, projects: ProjectRepository, requests: QuoteRequestRepository, quotes: QuoteRepository) -> None`		45–55
`def build(period: str = 'daily', buckets: int = DEFAULT_BUCKETS, now: datetime | None = None) -> Report`	Return the last ``buckets`` periods, oldest first.	57–85
`def _count(records: list, start: datetime, end: datetime) -> int`		88–89
`def _windows(period: str, buckets: int, now: datetime) -> list[tuple[datetime, datetime]]`	Return ``buckets`` windows, oldest first, ending at ``now``.	92–102
`def _floor(period: str, moment: datetime) -> datetime`		105–113
`def _shift(period: str, moment: datetime, count: int) -> datetime`		116–124
`def _label(period: str, start: datetime) -> str`		127–133

lines 42–133

```
# backend/app/services/report_service.py — lines 42–133, signatures only
class ReportService:
    """Builds reports from the four stores that hold timestamps."""
    def __init__(accounts: AccountRepository, projects: ProjectRepository, requests: QuoteRequestRepository, quotes:
QuoteRepository) -> None
    def build(period: str = 'daily', buckets: int = DEFAULT_BUCKETS, now: datetime | None = None) -> Report
        """Return the last ``buckets`` periods, oldest first."""
    @staticmethod
    def _count(records: list, start: datetime, end: datetime) -> int
    @staticmethod
    def _windows(period: str, buckets: int, now: datetime) -> list[tuple[datetime, datetime]]
        """Return ``buckets`` windows, oldest first, ending at ``now``."""
    @staticmethod
    def _floor(period: str, moment: datetime) -> datetime
    @staticmethod
    def _shift(period: str, moment: datetime, count: int) -> datetime
    @staticmethod
    def _label(period: str, start: datetime) -> str
```

4.26 ShortlistService

backend/app/services/shortlist_service.py · lines 14–72

Keeps the saved list and the project counters in step.

Method	Purpose	Lines
`def __init__(shortlists: ShortlistRepository, projects: ProjectRepository) -> None`		17–23
`def add(actor: Account, project_id: str) -> Project`	Save a project for the caller. Saving twice changes nothing.	25–34
`def remove(actor: Account, project_id: str) -> None`	Drop a project from the caller's list.	36–45
`def list_for(actor: Account) -> list[Project]`	Return the saved projects, oldest save first, skipping deleted ones.	47–54
`def ids_for(actor: Account) -> list[str]`	Return the saved project ids.	56–60
`def _own_list(actor: Account) -> Shortlist`		62–67

def _require_customer(actor: Account) -> None		70–72

lines 14–72

```
# backend/app/services/shortlist_service.py — lines 14–72, signatures only
class ShortlistService:
    """Keeps the saved list and the project counters in step."""
    def __init__(shortlists: ShortlistRepository, projects: ProjectRepository) -> None
    def add(actor: Account, project_id: str) -> Project
        """Save a project for the caller. Saving twice changes nothing."""
    def remove(actor: Account, project_id: str) -> None
        """Drop a project from the caller's list."""
    def list_for(actor: Account) -> list[Project]
        """Return the saved projects, oldest save first, skipping deleted ones."""
    def ids_for(actor: Account) -> list[str]
        """Return the saved project ids."""
    def _own_list(actor: Account) -> Shortlist
    @staticmethod
    def _require_customer(actor: Account) -> None
```

4.27 PaletteEntry

backend/app/services/tag_suggestion_service.py · lines 36–40

One colour of the photo and how much of it the photo holds.

lines 36–40

```
class PaletteEntry:
    """One colour of the photo and how much of it the photo holds."""


    hex: str
    percent: float
```

4.28 TagSuggestion

backend/app/services/tag_suggestion_service.py · lines 44–52

What the analyser found. The designer accepts or ignores it.

lines 44–52

```
class TagSuggestion:
    """What the analyser found. The designer accepts or ignores it."""


    palette: list[PaletteEntry]
    tags: list[str]
    width: int
    height: int
    subject: str = "room"
    objects: list[str] = field(default_factory=list)
```

4.29 suggest_tags()

backend/app/services/tag_suggestion_service.py · lines 55–82

Read ``data`` as an image and describe it.

lines 55–82

```
def suggest_tags(data: bytes) -> TagSuggestion:
    """Read ``data`` as an image and describe it.


    Raises ``ValueError`` when the bytes are not an image Pillow can read.
    A photo the model reads as an animal, a person, a vehicle or a landscape is
    returned with ``subject="other"`` and no tags at all: half a tag list is
    worse than a clear refusal, because the tags would drive a search.
    """
    try:
        with Image.open(BytesIO(data)) as image:
            image.load()
            width, height = image.size
            small = image.convert("RGB").resize(ANALYSIS_SIZE)
            palette = _palette(small)
            stats = _stats(small)
    except (OSError, ValueError) as exc:
        raise ValueError("The photo could not be read as an image") from exc
    reading = read_objects(data)
    subject = reading.subject if reading is not None else "room"
    objects = reading.tags if reading is not None else []
    return TagSuggestion(
        palette=palette,
        tags=[] if subject == "other" else _merge(objects, _tags(stats)),
        width=width,
        height=height,
        subject=subject,
        objects=[] if subject == "other" else objects,
    )
```

4.30 _merge()

backend/app/services/tag_suggestion_service.py · lines 85–91

Object tags first, then mood tags, without repeats, capped.

lines 85–91

```
def _merge(objects: list[str], moods: list[str]) -> list[str]:
    """Object tags first, then mood tags, without repeats, capped."""
    merged: list[str] = []
    for tag in (*objects, *moods):
        if tag not in merged:
            merged.append(tag)
    return merged[:TAG_LIMIT]
```

4.31 _palette()

backend/app/services/tag_suggestion_service.py · lines 94–109

lines 94–109

```
def _palette(image: Image.Image) -> list[PaletteEntry]:
    quantized = image.quantize(colors=PALETTE_SIZE, method=Image.Quantize.MEDIANCUT)
    table = quantized.getpalette() or []
    counts = quantized.getcolors() or []
    total = sum(count for count, _ in counts) or 1
    entries: list[PaletteEntry] = []
    for count, index in sorted(counts, reverse=True):
        base = index * 3
        colour = tuple(table[base : base + 3])
        entries.append(
            PaletteEntry(
                hex="#%02x%02x%02x" % colour,
                percent=round(100 * count / total, 1),
            )
        )
    return entries
```

4.32 _stats()

backend/app/services/tag_suggestion_service.py · lines 112–124

lines 112–124

```
def _stats(image: Image.Image) -> dict[str, float]:
    mean = ImageStat.Stat(image)
    brightness = sum(mean.mean) / 3
    contrast = sum(mean.stddev) / 3
    saturation = ImageStat.Stat(image.convert("HSV")).mean[1]
    grey = image.convert("L")
    edge = ImageStat.Stat(grey.filter(ImageFilter.FIND_EDGES)).mean[0]
    return {
        "brightness": brightness,
        "contrast": contrast,
        "saturation": saturation,
        "edge": edge,
    }
```

4.33 _tags()

backend/app/services/tag_suggestion_service.py · lines 127–150

lines 127–150

```
def _tags(stats: dict[str, float]) -> list[str]:
    tags: list[str] = []

    if stats["brightness"] >= BRIGHT_MEAN:
        tags.append("bright")
    elif stats["brightness"] <= MOODY_MEAN:
        tags.append("moody")
    else:
        tags.append("balanced light")

    if stats["saturation"] <= NEUTRAL_SATURATION:
        tags.append("neutral palette")
    elif stats["saturation"] >= SATURATED_SATURATION:
        tags.append("saturated palette")
    else:
        tags.append("warm palette")

    if stats["contrast"] <= FLAT_CONTRAST:
        tags.append("soft contrast")
    elif stats["contrast"] >= HIGH_CONTRAST:
        tags.append("high contrast")

    tags.append("calm detail" if stats["edge"] < CALM_EDGE else "busy detail")
    return tags
```

5 Control boundary: backend/app/api

A router function is the HTTP edge. It validates the body with a schema, asks the service to do the work, and translates a domain error into a status code.

5.1 _to_http()

backend/app/api/admin.py · lines 22–26

Map a domain refusal to the matching response.

lines 22–26

```
def _to_http(exc: Exception) -> HTTPException:
    """Map a domain refusal to the matching response."""
    if isinstance(exc, AccountNotFound):
        return HTTPException(status_code=http_status.HTTP_404_NOT_FOUND, detail=exc.message)
    return HTTPException(status_code=http_status.HTTP_403_FORBIDDEN, detail=exc.message)
```

5.2 list_accounts()

backend/app/api/admin.py · lines 34–44

@router.get('/accounts', response_model=list[AccountPublic], summary='List accounts for an administrator')

Return every account, optionally filtered by status.

lines 34–44

```
def list_accounts(
    actor: CurrentAccountDep,
    service: ServiceDep,
    account_status: Annotated[AccountStatus | None, Query(alias="status")] = None,
) -> list[AccountPublic]:
    """Return every account, optionally filtered by status."""
    try:
        accounts = service.list_accounts(actor, account_status)
    except NotPermitted as exc:
        raise _to_http(exc) from exc
    return [AccountPublic.model_validate(account) for account in accounts]
```

5.3 approve_account()

backend/app/api/admin.py · lines 52–60

@router.post('/accounts/{account_id}/approve', response_model=AccountPublic, summary='Approve an account')

Activate the account so it may sign in.

lines 52–60

```
def approve_account(
    account_id: str, actor: CurrentAccountDep, service: ServiceDep
) -> AccountPublic:
    """Activate the account so it may sign in."""
    try:
        account = service.approve(actor, account_id)
    except (AccountNotFound, NotPermitted) as exc:
        raise _to_http(exc) from exc
    return AccountPublic.model_validate(account)
```

5.4 suspend_account()

backend/app/api/admin.py · lines 68–76

@router.post('/accounts/{account_id}/suspend', response_model=AccountPublic, summary='Suspend an account')

Suspend the account so it cannot sign in again.

lines 68–76

```
def suspend_account(
    account_id: str, actor: CurrentAccountDep, service: ServiceDep
) -> AccountPublic:
    """Suspend the account so it cannot sign in again."""
    try:
        account = service.suspend(actor, account_id)
    except (AccountNotFound, NotPermitted) as exc:
        raise _to_http(exc) from exc
    return AccountPublic.model_validate(account)
```

5.5 register()

backend/app/api/auth.py · lines 40–55

@router.post('/register', response_model=AccountPublic, status_code=status.HTTP_201_CREATED, summary='Register a customer or designer account')

Create a self service account.

lines 40–55

```
def register(payload: RegisterRequest, service: ServiceDep) -> AccountPublic:
    """Create a self service account."""
    try:
        account = service.register(
            email=payload.email,
            password=payload.password,
            display_name=payload.display_name,
            role=payload.role,
        )
    except EmailAlreadyRegistered as exc:
        raise HTTPException(status_code=status.HTTP_409_CONFLICT, detail=exc.message) from exc
    except (InvalidRoleForSelfRegistration, WeakPassword) as exc:
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST, detail=exc.message
        ) from exc
    return AccountPublic.model_validate(account)
```

5.6 login()

backend/app/api/auth.py · lines 59–76

@router.post('/login', response_model=TokenResponse, summary='Exchange credentials for a token')

Authenticate an account and issue an access token.

lines 59–76

```
def login(payload: LoginRequest, service: ServiceDep) -> TokenResponse:
    """Authenticate an account and issue an access token."""
    try:
        account = service.authenticate(payload.email, payload.password)
    except InvalidCredentials as exc:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED, detail=exc.message
        ) from exc
    except AccountNotActive as exc:
        raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail=exc.message) from exc
    minutes = get_token_minutes()
    token = create_access_token(account.id, account.role.value, minutes)
    return TokenResponse(
        access_token=token,
        token_type="bearer",
        expires_in=minutes * 60,
        account=AccountPublic.model_validate(account),
    )
```

5.7 read_current_account()

backend/app/api/auth.py · lines 80–82

@router.get('/me', response_model=AccountPublic, summary='Return the authenticated account')

Return the account the bearer token belongs to.

lines 80–82

```
def read_current_account(account: CurrentAccountDep) -> AccountPublic:
    """Return the account the bearer token belongs to."""
    return AccountPublic.model_validate(account)
```

5.8 BudgetRange

backend/app/api/catalogue.py · lines 17–22

The slider bounds for the budget filter.

lines 17–22

```
class BudgetRange(BaseModel):
    """The slider bounds for the budget filter."""


    min: int = Field(description="Lowest selectable amount")
    max: int = Field(description="Highest selectable amount")
    step: int = Field(description="Slider step in the same currency unit")
```

5.9 CatalogueResponse

backend/app/api/catalogue.py · lines 25–30

Everything a filter screen needs to build its controls.

lines 25–30

```
class CatalogueResponse(BaseModel):
    """Everything a filter screen needs to build its controls."""


    categories: list[str]
    styles: list[str]
    budget: BudgetRange
```

5.10 read_catalogue()

backend/app/api/catalogue.py · lines 38–50

@router.get('', response_model=CatalogueResponse, summary='Read the fixed category and style lists')

Return the platform vocabulary. No sign in needed.

lines 38–50

```
def read_catalogue(
    categories: Annotated[CategoryRepository, Depends(get_category_repository)],
) -> CatalogueResponse:
    """Return the platform vocabulary. No sign in needed."""
    return CatalogueResponse(
        categories=[entry.name for entry in categories.list_active()],
        styles=list(catalogue.STYLES),
        budget=BudgetRange(
            min=catalogue.BUDGET_MIN,
            max=catalogue.BUDGET_MAX,
            step=catalogue.BUDGET_STEP,
        ),
    )
```

5.11 CategoryPublic

backend/app/api/categories.py · lines 25–32

One category as the screens read it.

lines 25–32

```
class CategoryPublic(BaseModel):
    """One category as the screens read it."""


    id: str
    name: str
    active: bool
    created_at: str
    updated_at: str
```

5.12 CategoryInput

backend/app/api/categories.py · lines 35–38

Payload to create or rename a category.

lines 35–38

```
class CategoryInput(BaseModel):
    """Payload to create or rename a category."""

    name: str = Field(min_length=1, max_length=60)
```

5.13 CategoryPatch

backend/app/api/categories.py · lines 41–45

Payload to rename, archive or restore.

lines 41–45

```
class CategoryPatch(BaseModel):
    """Payload to rename, archive or restore."""

    name: str | None = Field(default=None, min_length=1, max_length=60)
    active: bool | None = None
```

5.14 _to_public()

backend/app/api/categories.py · lines 48–55

lines 48–55

```
def _to_public(category) -> CategoryPublic:
    return CategoryPublic(
        id=category.id,
        name=category.name,
        active=category.active,
        created_at=category.created_at.isoformat(),
        updated_at=category.updated_at.isoformat() if category.updated_at else "",
    )
```

5.15 _fail()

backend/app/api/categories.py · lines 58–65

lines 58–65

```
def _fail(exc: Exception) -> HTTPException:
    if isinstance(exc, CategoryNotFound):
        return HTTPException(status_code=http_status.HTTP_404_NOT_FOUND, detail=str(exc))
    if isinstance(exc, CategoryExists):
        return HTTPException(status_code=http_status.HTTP_409_CONFLICT, detail=str(exc))
    if isinstance(exc, NotPermitted):
        return HTTPException(status_code=http_status.HTTP_403_FORBIDDEN, detail=exc.message)
    return HTTPException(status_code=http_status.HTTP_400_BAD_REQUEST, detail=str(exc))
```

5.16 list_categories()

backend/app/api/categories.py · lines 73–75

@router.get('', response_model=list[CategoryPublic], summary='List the categories a dropdown should show')

Public list of active categories.

lines 73–75

```
def list_categories(service: ServiceDep) -> list[CategoryPublic]:
    """Public list of active categories."""
    return [_to_public(category) for category in service.list_active()]
```

5.17 list_all_categories()

backend/app/api/categories.py · lines 83–91

@router.get('/all', response_model=list[CategoryPublic], summary='List every category, archived ones included')

Staff only.

lines 83–91

```
def list_all_categories(
    actor: CurrentAccountDep, service: ServiceDep
) -> list[CategoryPublic]:
    """Staff only."""
    try:
        categories = service.list_all(actor)
    except NotPermitted as exc:
        raise _fail(exc) from exc
    return [_to_public(category) for category in categories]
```

5.18 create_category()

backend/app/api/categories.py · lines 100–108

@router.post('', response_model=CategoryPublic, status_code=http_status.HTTP_201_CREATED, summary='Create a category')

Staff only.

lines 100–108

```
def create_category(
    payload: CategoryInput, actor: CurrentAccountDep, service: ServiceDep
) -> CategoryPublic:
    """Staff only."""
    try:
        category = service.add(actor, payload.name)
    except (NotPermitted, CategoryExists, ValueError) as exc:
        raise _fail(exc) from exc
    return _to_public(category)
```

5.19 update_category()

backend/app/api/categories.py · lines 116–132

@router.patch('/{category_id}', response_model=CategoryPublic, summary='Rename, archive or restore a category')

Staff only.

lines 116–132

```
def update_category(
    category_id: str,
    payload: CategoryPatch,
    actor: CurrentAccountDep,
    service: ServiceDep,
) -> CategoryPublic:
    """Staff only."""
    try:
        if payload.name is not None:
            service.rename(actor, category_id, payload.name)
        if payload.active is not None:
            category = service.set_active(actor, category_id, payload.active)
        else:
            category = service.get(actor, category_id)
    except (NotPermitted, CategoryNotFound, CategoryExists, ValueError) as exc:
        raise _fail(exc) from exc
    return _to_public(category)
```

5.20 _to_http()

backend/app/api/companies.py · lines 21–31

Map a domain refusal to the matching response.

lines 21–31

```
def _to_http(exc: Exception) -> HTTPException:
    """Map a domain refusal to the matching response."""
    if isinstance(exc, CompanyNotFound):
        return HTTPException(status_code=http_status.HTTP_404_NOT_FOUND, detail=exc.message)
    if isinstance(exc, CompanyAlreadyExists):
        return HTTPException(status_code=http_status.HTTP_409_CONFLICT, detail=exc.message)
    if isinstance(exc, ValueError):
        return HTTPException(
            status_code=http_status.HTTP_400_BAD_REQUEST, detail=str(exc)
        )
    return HTTPException(status_code=http_status.HTTP_403_FORBIDDEN, detail=exc.message)
```

5.21 create_company()

backend/app/api/companies.py · lines 40–48

@router.post('', response_model=CompanyPublic, status_code=http_status.HTTP_201_CREATED, summary='Create the company profile of the signed in designer')

Create the one profile this designer may hold.

lines 40–48

```
def create_company(
    payload: CompanyCreate, actor: CurrentAccountDep, service: ServiceDep
) -> CompanyPublic:
    """Create the one profile this designer may hold."""
    try:
        company = service.create(actor, **payload.model_dump())
    except (NotPermitted, CompanyAlreadyExists, ValueError) as exc:
        raise _to_http(exc) from exc
    return CompanyPublic.model_validate(company)
```

5.22 read_my_company()

backend/app/api/companies.py · lines 56–62

@router.get('/me', response_model=CompanyPublic, summary='Read the company profile of the signed in designer')

Return the caller's own profile.

lines 56–62

```
def read_my_company(actor: CurrentAccountDep, service: ServiceDep) -> CompanyPublic:
    """Return the caller's own profile."""
    try:
        company = service.get_for_owner(actor)
    except (NotPermitted, CompanyNotFound) as exc:
        raise _to_http(exc) from exc
    return CompanyPublic.model_validate(company)
```

5.23 update_my_company()

backend/app/api/companies.py · lines 70–79

@router.patch('/me', response_model=CompanyPublic, summary="Change fields of the signed in designer's profile")

Write only the fields that were supplied.

lines 70–79

```
def update_my_company(
    payload: CompanyUpdate, actor: CurrentAccountDep, service: ServiceDep
) -> CompanyPublic:
    """Write only the fields that were supplied."""
    changes = payload.model_dump(exclude_unset=True)
    try:
        company = service.update(actor, **changes)
    except (NotPermitted, CompanyNotFound, ValueError) as exc:
        raise _to_http(exc) from exc
    return CompanyPublic.model_validate(company)
```

5.24 list_companies()

backend/app/api/companies.py · lines 87–89

@router.get('', response_model=list[CompanyPublic], summary='List every company profile')

Return the published profiles so a customer can browse them.

lines 87–89

```
def list_companies(actor: CurrentAccountDep, service: ServiceDep) -> list[CompanyPublic]:
    """Return the published profiles so a customer can browse them."""
    return [CompanyPublic.model_validate(company) for company in service.list_companies()]
```

5.25 read_company()

backend/app/api/companies.py · lines 97–105

@router.get('/{company_id}', response_model=CompanyPublic, summary='Read any company profile by id')

Return a profile. Any signed in account may read it.

lines 97–105

```
def read_company(
    company_id: str, actor: CurrentAccountDep, service: ServiceDep
) -> CompanyPublic:
    """Return a profile. Any signed in account may read it."""
    try:
        company = service.get(company_id)
    except CompanyNotFound as exc:
        raise _to_http(exc) from exc
    return CompanyPublic.model_validate(company)
```

5.26 get_account_repository()

backend/app/api/deps.py · lines 45–47

Return the process wide account store.

lines 45–47

```
def get_account_repository() -> AccountRepository:
    """Return the process wide account store."""
    return _account_repository
```

5.27 get_account_service()

backend/app/api/deps.py · lines 50–52

Return a service bound to the shared account store.

lines 50–52

```
def get_account_service() -> AccountService:
    """Return a service bound to the shared account store."""
    return AccountService(get_account_repository())
```

5.28 get_company_repository()

backend/app/api/deps.py · lines 55–57

Return the process wide company store.

lines 55–57

```
def get_company_repository() -> CompanyRepository:
    """Return the process wide company store."""
    return _company_repository
```

5.29 get_company_service()

backend/app/api/deps.py · lines 60–62

Return a service bound to the shared company store.

lines 60–62

```
def get_company_service() -> CompanyService:
    """Return a service bound to the shared company store."""
    return CompanyService(get_company_repository())
```

5.30 get_project_repository()

backend/app/api/deps.py · lines 65–67

Return the process wide project store.

lines 65–67

```
def get_project_repository() -> ProjectRepository:
    """Return the process wide project store."""
    return _project_repository
```

5.31 get_project_service()

backend/app/api/deps.py · lines 70–72

Return a service bound to the shared project and company stores.

lines 70–72

```
def get_project_service() -> ProjectService:
    """Return a service bound to the shared project and company stores."""
    return ProjectService(get_project_repository(), get_company_repository())
```

5.32 get_project_search_service()

backend/app/api/deps.py · lines 75–77

Return a search service bound to the shared project store.

lines 75–77

```
def get_project_search_service() -> ProjectSearchService:
    """Return a search service bound to the shared project store."""
    return ProjectSearchService(get_project_repository())
```

5.33 get_shortlist_repository()

backend/app/api/deps.py · lines 80–82

Return the process wide shortlist store.

lines 80–82

```
def get_shortlist_repository() -> ShortlistRepository:
    """Return the process wide shortlist store."""
    return _shortlist_repository
```

5.34 get_shortlist_service()

backend/app/api/deps.py · lines 85–87

Return a service bound to the shared shortlist and project stores.

lines 85–87

```
def get_shortlist_service() -> ShortlistService:
    """Return a service bound to the shared shortlist and project stores."""
    return ShortlistService(get_shortlist_repository(), get_project_repository())
```

5.35 get_quote_request_repository()

backend/app/api/deps.py · lines 90–92

Return the process wide quote request store.

lines 90–92

```
def get_quote_request_repository() -> QuoteRequestRepository:
    """Return the process wide quote request store."""
    return _quote_request_repository
```

5.36 get_quote_repository()

backend/app/api/deps.py · lines 95–97

Return the process wide quote store.

lines 95–97

```
def get_quote_repository() -> QuoteRepository:
    """Return the process wide quote store."""
    return _quote_repository
```

5.37 get_quote_service()

backend/app/api/deps.py · lines 100–107

Return a service bound to the shared quote, company and project stores.

lines 100–107

```
def get_quote_service() -> QuoteService:
    """Return a service bound to the shared quote, company and project stores."""
    return QuoteService(
        get_quote_request_repository(),
        get_quote_repository(),
        get_company_repository(),
        get_project_repository(),
    )
```

5.38 get_category_repository()

backend/app/api/deps.py · lines 110–112

Return the process wide category store.

lines 110–112

```
def get_category_repository() -> CategoryRepository:
    """Return the process wide category store."""
    return _category_repository
```

5.39 get_category_service()

backend/app/api/deps.py · lines 115–117

Return a service bound to the shared category store.

lines 115–117

```
def get_category_service() -> CategoryService:
    """Return a service bound to the shared category store."""
    return CategoryService(get_category_repository())
```

5.40 get_report_service()

backend/app/api/deps.py · lines 120–127

Return a report service bound to the four stores that hold timestamps.

lines 120–127

```
def get_report_service() -> ReportService:
    """Return a report service bound to the four stores that hold timestamps."""
    return ReportService(
        get_account_repository(),
        get_project_repository(),
        get_quote_request_repository(),
        get_quote_repository(),
    )
```

5.41 get_current_account()

backend/app/api/deps.py · lines 130–164

Return the account named by the bearer token.

lines 130–164

```
# backend/app/api/deps.py — lines 130–164, first 20 lines of 35
def get_current_account(
    credentials: Annotated[HTTPAuthorizationCredentials | None, Depends(_bearer_scheme)],
    service: Annotated[AccountService, Depends(get_account_service)],
) -> Account:
    """Return the account named by the bearer token.

    Raises 401 for a missing, malformed, expired or orphaned token.
    """
    if credentials is None:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Not authenticated",
            headers={"WWW-Authenticate": "Bearer"},
        )
    try:
        claims = decode_access_token(credentials.credentials)
    except ValueError as exc:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail=str(exc),
    ...
```

5.42 HealthResponse

backend/app/api/health.py · lines 16–21

Liveness payload returned by the platform.

lines 16–21

```
class HealthResponse(BaseModel):
    """Liveness payload returned by the platform."""

    status: Literal["ok"]
    version: str
    time: datetime
```

5.43 read_health()

backend/app/api/health.py · lines 25–31

@router.get('/health', response_model=HealthResponse, summary='Service liveness')

Report that the API is running.

lines 25–31

```
def read_health() -> HealthResponse:
    """Report that the API is running."""
    return HealthResponse(
        status="ok",
        version=APP_VERSION,
        time=datetime.now(timezone.utc),
    )
```

5.44 _read_photo()

backend/app/api/images.py · lines 58–78

Return the bytes of an allowed photo, or raise the refusal.

lines 58–78

```
async def _read_photo(file: UploadFile) -> bytes:
    """Return the bytes of an allowed photo, or raise the refusal."""
    declared = (file.content_type or "").lower()
    if declared and declared not in CONTENT_TYPES:
        raise HTTPException(
            status_code=http_status.HTTP_415_UNSUPPORTED_MEDIA_TYPE,
            detail="Upload a JPEG, PNG or WebP photo",
        )

    data = await file.read(MAX_UPLOAD_BYTES + 1)
    if len(data) > MAX_UPLOAD_BYTES:
        raise HTTPException(
            status_code=http_status.HTTP_413_REQUEST_ENTITY_TOO_LARGE,
            detail="The photo is larger than 5 MB",
        )
    if not data:
        raise HTTPException(
            status_code=http_status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail="The file is empty",
        )
    return data
```

5.45 store_upload()

backend/app/api/images.py · lines 86–114

@router.post('', response_model=StoredImagePublic, summary='Store an uploaded photo and return its address')

Keep a designer's photo so the project can show it later.

lines 86–114

```
async def store_upload(
    actor: CurrentAccountDep,
    file: UploadFile = File(...),
) -> StoredImagePublic:
    """Keep a designer's photo so the project can show it later.

    The file is checked the same way as the tag reader: Pillow decodes it, so a
    renamed text file is refused. The name is random, so two designers can both
    upload ``kitchen.jpg`` without meeting.
    """
    data = await _read_photo(file)

    try:
        with Image.open(io.BytesIO(data)) as probe:
            probe.verify()
        with Image.open(io.BytesIO(data)) as probe:
            width, height = probe.size
    except Exception as exc:  # Pillow raises several unrelated types
        raise HTTPException(
            status_code=http_status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail="That file is not an image",
        ) from exc

    declared = (file.content_type or "").lower()
    name = f"{uuid.uuid4().hex}{EXTENSIONS.get(declared, '.jpg')}"
    UPLOAD_ROOT.mkdir(parents=True, exist_ok=True)
    (UPLOAD_ROOT / name).write_bytes(data)

    return StoredImagePublic(url=f"/uploads/{name}", width=width, height=height)
```

5.46 tags_for_upload()

backend/app/api/images.py · lines 122–169

@router.post('/tags', response_model=TagSuggestionPublic, summary='Read an uploaded photo and return catalogue tags')

Describe an uploaded photo. Any signed-in account may call this.

lines 122–169

```
# backend/app/api/images.py — lines 122–169, first 20 lines of 48
async def tags_for_upload(
    actor: CurrentAccountDep,
    file: UploadFile = File(...),
) -> TagSuggestionPublic:
    """Describe an uploaded photo. Any signed-in account may call this.

    The bytes are checked by Pillow, not by the declared type alone: a renamed
    text file is refused with 422.
    """
    declared = (file.content_type or "").lower()
    if declared and declared not in CONTENT_TYPES:
        raise HTTPException(
            status_code=http_status.HTTP_415_UNSUPPORTED_MEDIA_TYPE,
            detail="Upload a JPEG, PNG or WebP photo",
        )

    data = await file.read(MAX_UPLOAD_BYTES + 1)
    if len(data) > MAX_UPLOAD_BYTES:
        raise HTTPException(
            status_code=http_status.HTTP_413_REQUEST_ENTITY_TOO_LARGE,
    ...
```

5.47 _to_http()

backend/app/api/projects.py · lines 43–51

Map a domain refusal to the matching response.

lines 43–51

```
def _to_http(exc: Exception) -> HTTPException:
    """Map a domain refusal to the matching response."""
    if isinstance(exc, ProjectNotFound):
        return HTTPException(status_code=http_status.HTTP_404_NOT_FOUND, detail=exc.message)
    if isinstance(exc, (CompanyNotFound, NotPermitted)):
        return HTTPException(status_code=http_status.HTTP_403_FORBIDDEN, detail=exc.message)
    if isinstance(exc, ValueError):
        return HTTPException(status_code=http_status.HTTP_400_BAD_REQUEST, detail=str(exc))
    return HTTPException(status_code=http_status.HTTP_400_BAD_REQUEST, detail=str(exc))
```

5.48 create_project()

backend/app/api/projects.py · lines 60–68

@router.post('', response_model=ProjectPublic, status_code=http_status.HTTP_201_CREATED, summary='Publish a project under the designer company')

Publish one portfolio project for the signed in designer.

lines 60–68

```
def create_project(
    payload: ProjectCreate, actor: CurrentAccountDep, service: ServiceDep
) -> ProjectPublic:
    """Publish one portfolio project for the signed in designer."""
    try:
        project = service.create(actor, **payload.model_dump())
    except (NotPermitted, CompanyNotFound, ValueError) as exc:
        raise _to_http(exc) from exc
    return ProjectPublic.model_validate(project)
```

5.49 list_my_projects()

backend/app/api/projects.py · lines 76–95

@router.get('/mine', response_model=list[ProjectPublic], summary="List the signed in designer's portfolio")

Return the owner's projects filtered by status and title.

lines 76–95

```
def list_my_projects(
    actor: CurrentAccountDep,
    service: ServiceDep,
    status: ProjectStatus | None = None,
    name: str = "",
) -> list[ProjectPublic]:
    """Return the owner's projects filtered by status and title."""
    try:
        projects = service.list_for_owner(
            actor,
            status=status,
            name=name,
        )
    except NotPermitted as exc:
        raise _to_http(exc) from exc

    return [
        ProjectPublic.model_validate(project)
        for project in projects
    ]
```

5.50 list_company_projects()

backend/app/api/projects.py · lines 102–107

@router.get('/company/{company_id}', response_model=list[ProjectPublic], summary='List the published projects of one company')

Return the portfolio a customer browsing a designer would see.

lines 102–107

```
def list_company_projects(
    company_id: str, actor: CurrentAccountDep, service: ServiceDep
) -> list[ProjectPublic]:
    """Return the portfolio a customer browsing a designer would see."""
    projects = service.list_for_company(company_id)
    return [ProjectPublic.model_validate(project) for project in projects]
```

5.51 search_projects()

backend/app/api/projects.py · lines 115–153

@router.get('/search', response_model=SearchResponse, summary='Search the project catalogue')

Return one page of projects that match the filters.

lines 115–153

```
# backend/app/api/projects.py — lines 115–153, first 20 lines of 39
def search_projects(
    actor: CurrentAccountDep,
    service: SearchDep,
    keyword: str = "",
    category: str = "",
    style: str = "",
    budget_min: int | None = None,
    budget_max: int | None = None,
    tags: str = "",
    page: int = 1,
    page_size: int = 12,
) -> SearchResponse:
    """Return one page of projects that match the filters.

    ``tags`` takes a comma separated list. A project must carry at least one of
    them, and the closest match comes first.
    """
    try:
        result = service.search(
            keyword=keyword,
    ...
```

5.52 read_project()

backend/app/api/projects.py · lines 161–169

@router.get('/{project_id}', response_model=ProjectPublic, summary='Read one project')

Return a project. Any signed in account may read it.

lines 161–169

```
def read_project(
    project_id: str, actor: CurrentAccountDep, service: ServiceDep
) -> ProjectPublic:
    """Return a project. Any signed in account may read it."""
    try:
        project = service.get(project_id)
    except ProjectNotFound as exc:
        raise _to_http(exc) from exc
    return ProjectPublic.model_validate(project)
```

5.53 update_project()

backend/app/api/projects.py · lines 177–189

@router.patch('/{project_id}', response_model=ProjectPublic, summary='Change fields of your own project')

Write only the supplied fields.

lines 177–189

```
def update_project(
    project_id: str,
    payload: ProjectUpdate,
    actor: CurrentAccountDep,
    service: ServiceDep,
) -> ProjectPublic:
    """Write only the supplied fields."""
    changes = payload.model_dump(exclude_unset=True)
    try:
        project = service.update(actor, project_id, **changes)
    except (ProjectNotFound, NotPermitted, ValueError) as exc:
        raise _to_http(exc) from exc
    return ProjectPublic.model_validate(project)
```

5.54 delete_project()

backend/app/api/projects.py · lines 197–205

@router.delete('/{project_id}', status_code=http_status.HTTP_204_NO_CONTENT, summary='Delete your own project')

Remove a project from the portfolio.

lines 197–205

```
def delete_project(
    project_id: str, actor: CurrentAccountDep, service: ServiceDep
) -> Response:
    """Remove a project from the portfolio."""
    try:
        service.delete(actor, project_id)
    except (ProjectNotFound, NotPermitted) as exc:
        raise _to_http(exc) from exc
    return Response(status_code=http_status.HTTP_204_NO_CONTENT)
```

5.55 record_project_view()

backend/app/api/projects.py · lines 213–222

@router.post('/{project_id}/view', status_code=http_status.HTTP_204_NO_CONTENT, summary='Count one look at a project')

Raise the view counter. The read endpoint stays free of side effects.

lines 213–222

```
def record_project_view(
    project_id: str, actor: CurrentAccountDep, service: ServiceDep
) -> Response:
    """Raise the view counter. The read endpoint stays free of side effects."""
    try:
        project = service.get(project_id)
    except ProjectNotFound as exc:
        raise _to_http(exc) from exc
    project.record_view()
    return Response(status_code=http_status.HTTP_204_NO_CONTENT)
```

5.56 suggest_project_tags()

backend/app/api/projects.py · lines 230–281

@router.post('/{project_id}/suggest-tags', response_model=TagSuggestionPublic, summary='Suggest description tags for one of your project photos')

Read one photo of your project and offer palette and mood tags.

lines 230–281

```
# backend/app/api/projects.py — lines 230–281, first 20 lines of 52
def suggest_project_tags(
    project_id: str,
    actor: CurrentAccountDep,
    service: ServiceDep,
    image_index: int = 0,
) -> TagSuggestionPublic:
    """Read one photo of your project and offer palette and mood tags.

    Nothing is saved. The designer accepts what fits and edits the rest.
    """
    try:
        project = service.require_owned(actor, project_id)
    except (ProjectNotFound, NotPermitted) as exc:
        raise _to_http(exc) from exc

    if not project.images:
        raise HTTPException(
            status_code=http_status.HTTP_400_BAD_REQUEST,
            detail="The project has no photo to analyse",
        )
    ...
```

5.57 _fail()

backend/app/api/quotes.py · lines 37–45

Map a domain refusal to the matching response.

lines 37–45

```
def _fail(exc: Exception) -> HTTPException:
    """Map a domain refusal to the matching response."""
    if isinstance(exc, (QuoteRequestNotFound, QuoteNotFound, CompanyNotFound, ProjectNotFound)):
        return HTTPException(status_code=http_status.HTTP_404_NOT_FOUND, detail=exc.message)
    if isinstance(exc, NotPermitted):
        return HTTPException(status_code=http_status.HTTP_403_FORBIDDEN, detail=exc.message)
    if isinstance(exc, InvalidState):
        return HTTPException(status_code=http_status.HTTP_409_CONFLICT, detail=exc.message)
    return HTTPException(status_code=http_status.HTTP_400_BAD_REQUEST, detail=str(exc))
```

5.58 _detail()

backend/app/api/quotes.py · lines 48–54

lines 48–54

```
def _detail(service: QuoteService, actor: Account, request_id: str) -> QuoteRequestDetail:
    request = service.get_request(actor, request_id)
    quotes = service.quotes_for(actor, request_id)
    return QuoteRequestDetail(
        **QuoteRequestPublic.model_validate(request).model_dump(),
        quotes=[QuotePublic.model_validate(quote) for quote in quotes],
    )
```

5.59 submit_request()

backend/app/api/quotes.py · lines 63–71

@router.post('/requests', response_model=QuoteRequestPublic, status_code=http_status.HTTP_201_CREATED, summary='Send a quote request to one company')

A customer asks one company for a price.

lines 63–71

```
def submit_request(
    payload: SubmitRequest, actor: CurrentAccountDep, service: ServiceDep
) -> QuoteRequestPublic:
    """A customer asks one company for a price."""
    try:
        request = service.submit(actor, **payload.model_dump())
    except (NotPermitted, CompanyNotFound, ProjectNotFound, InvalidState, ValueError) as exc:
        raise _fail(exc) from exc
    return QuoteRequestPublic.model_validate(request)
```

5.60 list_requests()

backend/app/api/quotes.py · lines 79–92

@router.get('/requests', response_model=list[QuoteRequestPublic], summary='List the requests you may see')

Customers see what they sent; designers see what their company received.

lines 79–92

```
def list_requests(
    actor: CurrentAccountDep,
    service: ServiceDep,
    state: QuoteRequestState | None = None,
) -> list[QuoteRequestPublic]:
    """Customers see what they sent; designers see what their company received."""
    try:
        if actor.role.value == "CUSTOMER":
            requests = service.list_for_customer(actor, state)
        else:
            requests = service.list_for_designer(actor, state)
    except (NotPermitted, CompanyNotFound) as exc:
        raise _fail(exc) from exc
    return [QuoteRequestPublic.model_validate(request) for request in requests]
```

5.61 read_request()

backend/app/api/quotes.py · lines 100–107

@router.get('/requests/{request_id}', response_model=QuoteRequestDetail, summary='Read one request with every quote version')

Return one request to a customer, the company, or platform staff.

lines 100–107

```
def read_request(
    request_id: str, actor: CurrentAccountDep, service: ServiceDep
) -> QuoteRequestDetail:
    """Return one request to a customer, the company, or platform staff."""
    try:
        return _detail(service, actor, request_id)
    except (QuoteRequestNotFound, NotPermitted) as exc:
        raise _fail(exc) from exc
```

5.62 counter_request()

backend/app/api/quotes.py · lines 115–126

@router.post('/requests/{request_id}/counter', response_model=QuoteRequestDetail, summary='Name a target price')

The customer records the price they want. It does not bind the designer.

lines 115–126

```
def counter_request(
    request_id: str,
    payload: CounterRequest,
    actor: CurrentAccountDep,
    service: ServiceDep,
) -> QuoteRequestDetail:
    """The customer records the price they want. It does not bind the designer."""
    try:
        service.counter(actor, request_id, payload.target_cents, payload.note)
        return _detail(service, actor, request_id)
    except (QuoteRequestNotFound, NotPermitted, InvalidState) as exc:
        raise _fail(exc) from exc
```

5.63 cancel_request()

backend/app/api/quotes.py · lines 134–145

@router.post('/requests/{request_id}/cancel', response_model=QuoteRequestPublic, summary='Cancel your request')

Stop an open request.

lines 134–145

```
def cancel_request(
    request_id: str,
    payload: ReasonRequest,
    actor: CurrentAccountDep,
    service: ServiceDep,
) -> QuoteRequestPublic:
    """Stop an open request."""
    try:
        request = service.cancel(actor, request_id, payload.reason)
    except (QuoteRequestNotFound, NotPermitted, InvalidState) as exc:
        raise _fail(exc) from exc
    return QuoteRequestPublic.model_validate(request)
```

5.64 refuse_request()

backend/app/api/quotes.py · lines 153–164

@router.post('/requests/{request_id}/refuse', response_model=QuoteRequestPublic, summary='Decline to quote')

The company declines the request.

lines 153–164

```
def refuse_request(
    request_id: str,
    payload: ReasonRequest,
    actor: CurrentAccountDep,
    service: ServiceDep,
) -> QuoteRequestPublic:
    """The company declines the request."""
    try:
        request = service.refuse(actor, request_id, payload.reason)
    except (QuoteRequestNotFound, NotPermitted, CompanyNotFound, InvalidState) as exc:
        raise _fail(exc) from exc
    return QuoteRequestPublic.model_validate(request)
```

5.65 answer_request()

backend/app/api/quotes.py · lines 173–184

@router.post('/requests/{request_id}/quotes', response_model=QuotePublic, status_code=http_status.HTTP_201_CREATED, summary='Answer a request with a total')

The company sends version one of the quote.

lines 173–184

```
def answer_request(
    request_id: str,
    payload: AnswerRequest,
    actor: CurrentAccountDep,
    service: ServiceDep,
) -> QuotePublic:
    """The company sends version one of the quote."""
    try:
        quote = service.answer(actor, request_id, **payload.model_dump())
    except (QuoteRequestNotFound, NotPermitted, CompanyNotFound, InvalidState) as exc:
        raise _fail(exc) from exc
    return QuotePublic.model_validate(quote)
```

5.66 list_quotes()

backend/app/api/quotes.py · lines 192–200

@router.get('/requests/{request_id}/quotes', response_model=list[QuotePublic], summary="List every version of a request's quote")

Return the versions, oldest first.

lines 192–200

```
def list_quotes(
    request_id: str, actor: CurrentAccountDep, service: ServiceDep
) -> list[QuotePublic]:
    """Return the versions, oldest first."""
    try:
        quotes = service.quotes_for(actor, request_id)
    except (QuoteRequestNotFound, NotPermitted) as exc:
        raise _fail(exc) from exc
    return [QuotePublic.model_validate(quote) for quote in quotes]
```

5.67 revise_quote()

backend/app/api/quotes.py · lines 209–220

@router.post('/{quote_id}/revise', response_model=QuotePublic, status_code=http_status.HTTP_201_CREATED, summary='Send the next version of a quote')

Replace the offered total with a new version.

lines 209–220

```
def revise_quote(
    quote_id: str,
    payload: AnswerRequest,
    actor: CurrentAccountDep,
    service: ServiceDep,
) -> QuotePublic:
    """Replace the offered total with a new version."""
    try:
        quote = service.revise(actor, quote_id, **payload.model_dump())
    except (QuoteNotFound, QuoteRequestNotFound, NotPermitted, InvalidState) as exc:
        raise _fail(exc) from exc
    return QuotePublic.model_validate(quote)
```

5.68 accept_quote()

backend/app/api/quotes.py · lines 228–236

@router.post('/{quote_id}/accept', response_model=QuotePublic, summary='Accept a quote')

Accept one quote. Other open requests stay open.

lines 228–236

```
def accept_quote(
    quote_id: str, actor: CurrentAccountDep, service: ServiceDep
) -> QuotePublic:
    """Accept one quote. Other open requests stay open."""
    try:
        quote = service.accept(actor, quote_id)
    except (QuoteNotFound, QuoteRequestNotFound, NotPermitted, InvalidState) as exc:
        raise _fail(exc) from exc
    return QuotePublic.model_validate(quote)
```

5.69 decline_quote()

backend/app/api/quotes.py · lines 244–255

@router.post('/{quote_id}/decline', response_model=QuotePublic, summary='Decline a quote')

Turn one quote down with an optional reason.

lines 244–255

```
def decline_quote(
    quote_id: str,
    payload: ReasonRequest,
    actor: CurrentAccountDep,
    service: ServiceDep,
) -> QuotePublic:
    """Turn one quote down with an optional reason."""
    try:
        quote = service.decline(actor, quote_id, payload.reason)
    except (QuoteNotFound, QuoteRequestNotFound, NotPermitted, InvalidState) as exc:
        raise _fail(exc) from exc
    return QuotePublic.model_validate(quote)
```

5.70 BucketPublic

backend/app/api/reports.py · lines 22–31

One period of activity.

lines 22–31

```
class BucketPublic(BaseModel):
    """One period of activity."""

    label: str
    start: str
    end: str
    accounts: int
    projects: int
    quote_requests: int
    quotes: int
```

5.71 ReportPublic

backend/app/api/reports.py · lines 34–38

A report with its period and buckets.

lines 34–38

```
class ReportPublic(BaseModel):
    """A report with its period and buckets."""

    period: str
    buckets: list[BucketPublic]
```

5.72 read_report()

backend/app/api/reports.py · lines 46–83

@router.get('', response_model=ReportPublic, summary='Read the activity report')

Staff only. Buckets run oldest first and end at now.

lines 46–83

```
# backend/app/api/reports.py — lines 46–83, first 20 lines of 38
def read_report(
    actor: CurrentAccountDep,
    service: ServiceDep,
    period: str = "daily",
    buckets: int = DEFAULT_BUCKETS,
) -> ReportPublic:
    """Staff only. Buckets run oldest first and end at now."""
    if actor.role not in PRIVILEGED_ROLES:
        raise HTTPException(
            status_code=http_status.HTTP_403_FORBIDDEN,
            detail="Only platform staff may read reports",
        )
    if buckets < 1 or buckets > MAX_BUCKETS:
        raise HTTPException(
            status_code=http_status.HTTP_400_BAD_REQUEST,
            detail=f"Buckets must sit between 1 and {MAX_BUCKETS}",
        )
    try:
        report = service.build(period=period, buckets=buckets)
    except ValueError as exc:
    ...
```

5.73 _forbidden()

backend/app/api/shortlist.py · lines 22–23

lines 22–23

```
def _forbidden(exc: NotPermitted) -> HTTPException:
    return HTTPException(status_code=http_status.HTTP_403_FORBIDDEN, detail=exc.message)
```

5.74 read_shortlist()

backend/app/api/shortlist.py · lines 31–41

@router.get('', response_model=ShortlistResponse, summary='Read the saved projects of the signed in customer')

Return the caller's saved projects, oldest save first.

lines 31–41

```
def read_shortlist(actor: CurrentAccountDep, service: ServiceDep) -> ShortlistResponse:
    """Return the caller's saved projects, oldest save first."""
    try:
        projects = service.list_for(actor)
        project_ids = service.ids_for(actor)
    except NotPermitted as exc:
        raise _forbidden(exc) from exc
    return ShortlistResponse(
        project_ids=project_ids,
        items=[ProjectPublic.model_validate(project) for project in projects],
    )
```

5.75 save_project()

backend/app/api/shortlist.py · lines 50–67

@router.post('/{project_id}', response_model=ShortlistResponse, status_code=http_status.HTTP_201_CREATED, summary='Save a project for the signed in customer')

Save one project. Saving a saved project changes nothing.

lines 50–67

```
def save_project(
    project_id: str, actor: CurrentAccountDep, service: ServiceDep
) -> ShortlistResponse:
    """Save one project. Saving a saved project changes nothing."""
    try:
        service.add(actor, project_id)
        projects = service.list_for(actor)
        project_ids = service.ids_for(actor)
    except NotPermitted as exc:
        raise _forbidden(exc) from exc
    except ProjectNotFound as exc:
        raise HTTPException(
            status_code=http_status.HTTP_404_NOT_FOUND, detail=exc.message
        ) from exc
    return ShortlistResponse(
        project_ids=project_ids,
        items=[ProjectPublic.model_validate(project) for project in projects],
    )
```

5.76 unsave_project()

backend/app/api/shortlist.py · lines 75–83

@router.delete('/{project_id}', status_code=http_status.HTTP_204_NO_CONTENT, summary="Drop a project from the signed in customer's shortlist")

Drop one project. Dropping an absent project changes nothing.

lines 75–83

```
def unsave_project(
    project_id: str, actor: CurrentAccountDep, service: ServiceDep
) -> Response:
    """Drop one project. Dropping an absent project changes nothing."""
    try:
        service.remove(actor, project_id)
    except NotPermitted as exc:
        raise _forbidden(exc) from exc
    return Response(status_code=http_status.HTTP_204_NO_CONTENT)
```

6 Data transfer shapes: backend/app/schemas

A schema is not a domain object. It states what the browser may send and what it receives. The validators in these classes reject a bad request before any use case runs.

Schema	File	Lines	Fields
AccountPublic	account.py	12–22	id, email, display_name, role, status, created_at
RegisterRequest	account.py	25–48	email, password, display_name, role
LoginRequest	account.py	51–55	email, password
TokenResponse	account.py	58–64	access_token, token_type, expires_in, account
CompanyPublic	company.py	10–24	id, owner_id, name, description, contact_email, contact_phone, locatio
CompanyCreate	company.py	27–35	name, description, contact_email, contact_phone, location, logo_url
CompanyUpdate	company.py	38–50	name, description, contact_email, contact_phone, location, logo_url
ProjectPublic	project.py	12–32	id, owner_id, company_id, title, description, style, category, budget_
ProjectCreate	project.py	35–45	title, description, style, category, budget_min, budget_max, images, t
ProjectUpdate	project.py	48–62	title, description, style, category, budget_min, budget_max, status, i
SearchResponse	project.py	65–72	items, total, page, page_size, pages
PaletteEntryPublic	project.py	75–79	hex, percent
TagSuggestionPublic	project.py	82–91	palette, tags, width, height, engine, subject, objects
StoredImagePublic	project.py	94–99	url, width, height
CounterPublic	quote.py	12–20	id, target_cents, note, created_at
QuotePublic	quote.py	23–39	id, request_id, version, total_cents, gst_included, gst_rate_bp, gst_c
QuoteRequestPublic	quote.py	42–60	id, customer_id, company_id, project_id, title, scope, budget_min_cent
QuoteRequestDetail	quote.py	63–66	quotes
SubmitRequest	quote.py	69–78	company_id, title, scope, budget_min_cents, budget_max_cents, target_c
CounterRequest	quote.py	81–85	target_cents, note
AnswerRequest	quote.py	88–94	total_cents, valid_days, gst_included, note
ReasonRequest	quote.py	97–100	reason
ShortlistResponse	shortlist.py	10–14	project_ids, items

The matching front-end types live in frontend/src/types.ts. They mirror these shapes and add the enum values the browser needs.

```
frontend/src/types.ts — the interfaces the API answers with
```

```
// frontend/src/types.ts · lines 1–228
export type Role = 'CUSTOMER' | 'DESIGNER' | 'ADMIN';
export interface AccountPublic { ... }
export interface HealthResponse { ... }
export interface CompanyPublic { ... }
export interface CompanyInput { ... }
export interface ProjectPublic { ... }
export interface SearchResponse { ... }
export interface SearchParams { ... }
export interface BudgetRange { ... }
export interface Catalogue { ... }
export interface RegisterRequest { ... }
export interface LoginRequest { ... }
export interface TokenResponse { ... }
export interface ProjectInput { ... }
export interface TagSuggestion { ... }
export interface StoredImage { ... }
export interface ShortlistResponse { ... }
export interface QuoteCounter { ... }
export interface QuotePublic { ... }
export interface QuoteRequestPublic { ... }
export interface QuoteRequestDetail { ... }
export interface CategoryPublic { ... }
export interface ReportBucket { ... }
export interface Report { ... }
```

7 Boundary layer: the browser

Each page is a boundary object. It draws the screen, holds the form state and calls one function in api.ts per endpoint. The table under each page names the calls it makes, so a sequence diagram can draw the message from page to API.

Route	Component
/	Home
/login	Login
/register	Register
*	Navigate

7.1 App

frontend/src/App.tsx · line 22

Calls: no endpoint directly; it composes other components.

from line 22 · first 28 lines

```
export default function App() {
  const { account, loading, logout } = useAuth();

  return (
    <div className="shell">
      <header className="site-header">
        <div className="container header-inner">
          <Link className="brand" to="/">
            Interior Design Platform
          </Link>
          <nav className="nav" aria-label="Main">
            <NavLink to="/" end>
              Home
            </NavLink>
            {account !== null && (
              <>
                <NavLink to="/dashboard">Dashboard</NavLink>
                <NavLink to="/search">Find a project</NavLink>
                <NavLink to="/designers">Designers</NavLink>
                {account.role === 'CUSTOMER' && <NavLink to="/saved">Saved</NavLink>}
                {(account.role === 'CUSTOMER' || account.role === 'DESIGNER') && (
                  <NavLink to="/quotes">
                    {account.role === 'DESIGNER' ? 'Quote inbox' : 'Requests'}
                  </NavLink>
                )}
                {account.role === 'DESIGNER' && (
                  <>
                  <NavLink to="/profile">Company profile</NavLink>
                  <NavLink to="/projects/new">Create Project</NavLink>
```

7.2 AuthProvider

frontend/src/auth.tsx · line 25

Calls: ApiError, clearToken, getToken, login, me, register, setToken.

React hooks: useContext, useEffect, useState.

Function	Line
async function restoreSession() {	33

from line 25 · first 28 lines

```
export function AuthProvider({ children }: { children: ReactNode }) {
  const [account, setAccount] = useState<AccountPublic | null>(null);
  const [token, setTokenState] = useState<string | null>(() => getToken());
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    let cancelled = false;

    async function restoreSession() {
      if (getToken() === null) {
        setLoading(false);
        return;
      }
      try {
        const current = await me();
        if (cancelled) return;
        setAccount(current);
        setTokenState(getToken());
      } catch (error) {
        if (cancelled) return;
        // Only a rejected token ends the session. A network failure keeps the
        // token so the next reload can retry instead of forcing a new sign in.
        if (error instanceof ApiError && (error.status === 401 || error.status === 403)) {
          clearToken();
        }
        setTokenState(getToken());
        setAccount(null);
      } finally {
        if (!cancelled) setLoading(false);
```

7.3 ProtectedRoute

frontend/src/components/ProtectedRoute.tsx · line 11

Calls: no endpoint directly; it composes other components.

from line 11 · first 28 lines

```
export default function ProtectedRoute({ children, roles }: ProtectedRouteProps) {
  const { account, loading } = useAuth();
  const location = useLocation();

  if (loading) {
    return (
      <div className="page-state" role="status">
        Checking your session…
      </div>
    );
  }

  if (account === null) {
    return <Navigate to="/login" replace state={{ from: location.pathname }} />;
  }

  if (roles !== undefined && !roles.includes(account.role)) {
    return <Navigate to="/dashboard" replace />;
  }

  return <>{children}</>;
}
```

7.4 Home

frontend/src/pages/Home.tsx · line 19

Calls: ApiError, health.

React hooks: useEffect, useState.

Function	Line
function describeFailure(caught: unknown): string {	9

from line 19 · first 28 lines

```
export default function Home() {
  const { account } = useAuth();
  const [state, setState] = useState<HealthState>('loading');
  const [info, setInfo] = useState<HealthResponse | null>(null);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let cancelled = false;

    health()
      .then((result) => {
        if (cancelled) return;
        setInfo(result);
        setState('online');
      })
      .catch((caught: unknown) => {
        if (cancelled) return;
        setError(describeFailure(caught));
        setState('offline');
      });

    return () => {
      cancelled = true;
    };
  }, []);

  return (
    <div className="stack">
      <section className="card hero">
```

7.5 Login

frontend/src/pages/Login.tsx · line 7

Calls: ApiError.

React hooks: useNavigate, useState.

Function	Line
async function handleSubmit(event: FormEvent<HTMLFormElement>) {	21

from line 7 · first 28 lines

```
export default function Login() {
  const { account, loading, login } = useAuth();
  const navigate = useNavigate();
  const location = useLocation();
  const destination = (location.state as { from?: string } | null)?.from ?? '/dashboard';
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState<string | null>(null);
  const [submitting, setSubmitting] = useState(false);

  if (!loading && account !== null) {
    return <Navigate to={destination} replace />;
  }

  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setError(null);
    setSubmitting(true);
    try {
      await login({ email: email.trim(), password });
      navigate(destination, { replace: true });
    } catch (caught) {
      setError(caught instanceof ApiError ? caught.detail : 'Unable to sign in. Try again.');
    } finally {
      setSubmitting(false);
    }
  }

  return (
```

7.6 Register

frontend/src/pages/Register.tsx · line 8

Calls: ApiError.

React hooks: useState.

Function	Line
function handleRoleChange(event: ChangeEvent<HTMLSelectElement>) {	18
async function handleSubmit(event: FormEvent<HTMLFormElement>) {	25

from line 8 · first 28 lines

```
export default function Register() {
  const { register } = useAuth();
  const [email, setEmail] = useState('');
  const [displayName, setDisplayName] = useState('');
  const [password, setPassword] = useState('');
  const [role, setRole] = useState<RegistrationRole>('CUSTOMER');
  const [error, setError] = useState<string | null>(null);
  const [submitting, setSubmitting] = useState(false);
  const [created, setCreated] = useState(false);

  function handleRoleChange(event: ChangeEvent<HTMLSelectElement>) {
    const value = event.target.value;
    if (value === 'CUSTOMER' || value === 'DESIGNER') {
      setRole(value);
    }
  }

  async function handleSubmit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    setError(null);
    setSubmitting(true);
    try {
      await register({
        email: email.trim(),
        password,
        display_name: displayName.trim(),
        role,
      });
      setCreated(true);
```

7.7 Dashboard

frontend/src/pages/Dashboard.tsx · line 24

Calls: no endpoint directly; it composes other components.

React hooks: useNavigate.

Function	Line
function formatDate(value: string): string {	18
function handleLogout() {	32

from line 24 · first 28 lines

```
export default function Dashboard() {
  const { account, logout } = useAuth();
  const navigate = useNavigate();

  if (account === null) {
    return null;
  }

  function handleLogout() {
    logout();
    navigate('/login', { replace: true });
  }

  return (
    <div className="stack">
      <div className="page-head">
        <div>
          <h1>Dashboard</h1>
          <p className="muted">Signed in as {account.email}</p>
        </div>
        <button type="button" className="button button-secondary" onClick={handleLogout}>
          Log out
        </button>
      </div>

      {account.status === 'PENDING' && (
        <p className="alert alert-notice" role="status">
          Your account is waiting for admin approval. Designer accounts become active once an
          administrator reviews them.
```

7.8 Search

frontend/src/pages/Search.tsx · line 18

Calls: ApiError, getCatalogue, listCompanies, searchProjects, suggestTagsFromUpload.

React hooks: useEffect, useState.

Function	Line
function formatBudget(value: number): string {	14
function submit(event: FormEvent<HTMLFormElement>) {	91
function clear() {	96
async function analysePhoto(event: ChangeEvent<HTMLInputElement>) {	111

from line 18 · first 28 lines

```
export default function Search() {
  const [catalogue, setCatalogue] = useState<Catalogue | null>(null);
  const [designers, setDesigners] = useState<Record<string, string>>({});
  const [keyword, setKeyword] = useState('');
  const [category, setCategory] = useState('');
  const [style, setStyle] = useState('');
  const [low, setLow] = useState(0);
  const [high, setHigh] = useState(200000);
  const [result, setResult] = useState<SearchResponse | null>(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [tags, setTags] = useState<string[]>([]);
  const [uploading, setUploading] = useState(false);
  const [photoNote, setPhotoNote] = useState<string | null>(null);
  const [preview, setPreview] = useState<string | null>(null);


  useEffect(() => {
    let live = true;
    getCatalogue()
      .then((data) => {
        if (!live) return;
        setCatalogue(data);
        setLow(data.budget.min);
        setHigh(data.budget.max);
      })
      .catch(() => {
        if (live) setError('Cannot load the filters. Check that the backend is running.');
      });
    listCompanies()
```

7.9 ProjectDetail

frontend/src/pages/ProjectDetail.tsx · line 19

Calls: ApiError, getProject, getShortlist, listCompanies, recordView, saveProject, unsaveProject.

React hooks: useEffect, useParams, useRef, useState.

Function	Line
function formatBudget(value: number): string {	15
async function toggleSaved() {	75

from line 19 · first 28 lines

```
export default function ProjectDetail() {
  const { projectId } = useParams<{ projectId: string }>();
  const { account } = useAuth();
  const [project, setProject] = useState<ProjectPublic | null>(null);
  const [companies, setCompanies] = useState<CompanyPublic[]>([]);
  const [active, setActive] = useState(0);
  const [saved, setSaved] = useState(false);
  const [busy, setBusy] = useState(false);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const counted = useRef(false);

  useEffect(() => {
    if (!projectId) return;
    let live = true;
    setLoading(true);
    setError(null);

    getProject(projectId)
      .then((data) => {
        if (!live) return;
        setProject(data);
        setActive(0);
      })
      .catch((caught: unknown) => {
        if (!live) return;
        setError(caught instanceof ApiError ? caught.detail : 'Unable to load the project.');
      })
      .finally(() => {
```

7.10 Saved

frontend/src/pages/Saved.tsx · line 10

Calls: ApiError, getShortlist, unsaveProject.

React hooks: useEffect, useState.

Function	Line
function formatBudget(value: number): string {	6
async function remove(project: ProjectPublic) {	35

from line 10 · first 28 lines

```
export default function Saved() {
  const [items, setItems] = useState<ProjectPublic[]>([]);
  const [loading, setLoading] = useState(true);
  const [busy, setBusy] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let live = true;
    getShortlist()
      .then((data) => {
        if (live) setItems(data.items);
      })
      .catch((caught: unknown) => {
        if (live) {
          setError(caught instanceof ApiError ? caught.detail : 'Unable to load saved projects.');
        }
      })
      .finally(() => {
        if (live) setLoading(false);
      });
    return () => {
      live = false;
    };
  }, []);

  async function remove(project: ProjectPublic) {
    setBusy(project.id);
    setError(null);
    try {
```

7.11 Designers

frontend/src/pages/Designers.tsx · line 6

Calls: ApiError, listCompanies.

React hooks: useEffect, useState.

```
from line 6 · first 28 lines
```

```
export default function Designers() {
  const [companies, setCompanies] = useState<CompanyPublic[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let cancelled = false;

    listCompanies()
      .then((rows) => {
        if (!cancelled) setCompanies(rows);
      })
      .catch((caught: unknown) => {
        if (cancelled) return;
        setError(caught instanceof ApiError ? caught.detail : 'Unable to load designers.');
      })
      .finally(() => {
        if (!cancelled) setLoading(false);
      });

    return () => {
      cancelled = true;
    };
  }, []);

  return (
    <div className="stack">
      <div className="page-head">
        <div>
```

7.12 DesignerDetail

frontend/src/pages/DesignerDetail.tsx · line 7

Calls: ApiError, companyById.

React hooks: useEffect, useParams, useState.

from line 7 · first 28 lines

```
export default function DesignerDetail() {
  const { companyId } = useParams<{ companyId: string }>();
  const { account } = useAuth();
  const [company, setCompany] = useState<CompanyPublic | null>(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    if (companyId === undefined) {
      setError('No designer was named.');
      setLoading(false);
      return;
    }

    let cancelled = false;

    companyById(companyId)
      .then((found) => {
        if (!cancelled) setCompany(found);
      })
      .catch((caught: unknown) => {
        if (cancelled) return;
        setError(caught instanceof ApiError ? caught.detail : 'Unable to load the profile.');
      })
      .finally(() => {
        if (!cancelled) setLoading(false);
      });

    return () => {
```

7.13 RequestQuote

frontend/src/pages/RequestQuote.tsx · line 7

Calls: ApiError, listCompanies, submitQuoteRequest.

React hooks: useEffect, useNavigate, useParams, useState.

Function	Line
async function submit(event: FormEvent<HTMLFormElement>) {	31

from line 7 · first 28 lines

```
export default function RequestQuote() {
  const { companyId } = useParams<{ companyId: string }>();
  const navigate = useNavigate();
  const [company, setCompany] = useState<CompanyPublic | null>(null);
  const [title, setTitle] = useState('');
  const [scope, setScope] = useState('');
  const [budgetMin, setBudgetMin] = useState('20000');
  const [budgetMax, setBudgetMax] = useState('40000');
  const [target, setTarget] = useState('');
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let live = true;
    listCompanies()
      .then((rows) => {
        if (live) setCompany(rows.find((row) => row.id === companyId) ?? null);
      })
      .catch(() => undefined);
    return () => {
      live = false;
    };
  }, [companyId]);

  async function submit(event: FormEvent<HTMLFormElement>) {
    event.preventDefault();
    if (!companyId || busy) return;
    setBusy(true);
    setError(null);
```

7.14 Quotes

frontend/src/pages/Quotes.tsx · line 8

Calls: ApiError, listQuoteRequests.

React hooks: useEffect, useState.

from line 8 · first 28 lines

```
export default function Quotes() {
  const { account } = useAuth();
  const [requests, setRequests] = useState<QuoteRequestPublic[]>([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);

  useEffect(() => {
    let live = true;
    listQuoteRequests()
      .then((data) => {
        if (live) setRequests(data);
      })
      .catch((caught: unknown) => {
        if (live) {
          setError(caught instanceof ApiError ? caught.detail : 'Unable to load requests.');
        }
      })
      .finally(() => {
        if (live) setLoading(false);
      });
    return () => {
      live = false;
    };
  }, []);

  const isDesigner = account?.role === 'DESIGNER';

  if (loading) return <p className="page-state">Loading requests…</p>;
```

7.15 QuoteDetail

frontend/src/pages/QuoteDetail.tsx · line 19

Calls: ApiError, acceptQuote, answerQuoteRequest, cancelQuoteRequest, counterQuoteRequest, declineQuote, getQuoteRequest, refuseQuoteRequest, reviseQuote.

React hooks: useEffect, useParams, useState.

Function	Line
async function run(action: () => Promise<unknown>, message: string) {	58
function centsFromInput(value: string): number {	73
function submitAnswer(event: FormEvent<HTMLFormElement>, reviseId?: string) {	77

from line 19 · first 28 lines

```
export default function QuoteDetail() {
  const { requestId } = useParams<{ requestId: string }>();
  const { account } = useAuth();
  const [request, setRequest] = useState<QuoteRequestDetail | null>(null);
  const [loading, setLoading] = useState(true);
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [notice, setNotice] = useState<string | null>(null);

  const [total, setTotal] = useState('');
  const [validDays, setValidDays] = useState('14');
  const [gstIncluded, setGstIncluded] = useState(false);
  const [note, setNote] = useState('');
  const [target, setTarget] = useState('');
  const [counterNote, setCounterNote] = useState('');
  const [reason, setReason] = useState('');

  const load = useCallback(async () => {
    if (!requestId) return;
    setError(null);
    try {
      const data = await getQuoteRequest(requestId);
      setRequest(data);
      const current = data.quotes.find((entry) => entry.state === 'OFFERED');
      if (current) {
        setTotal((current.total_cents / 100).toString());
        setGstIncluded(current.gst_included);
      }
    } catch (caught) {
```

7.16 MyProjects

frontend/src/pages/MyProjects.tsx · line 12

Calls: ApiError, deleteProject, myProjects, updateProject.

React hooks: useEffect, useState.

Function	Line
async function handleDelete(project: ProjectPublic) {	62
async function handleStatusChange(project: ProjectPublic) {	94

from line 12 · first 28 lines

```
export default function MyProjects() {
  const [projects, setProjects] = useState<ProjectPublic[]>([]);

  // Filters
  const [searchName, setSearchName] = useState('');
  const [statusFilter, setStatusFilter] =
    useState<ProjectStatus | ''>('');

  // Page status
  const [loading, setLoading] = useState(true);
  const [busyId, setBusyId] = useState<string | null>(null);
  const [error, setError] = useState<string | null>(null);
  const [notice, setNotice] = useState<string | null>(null);

  // Retrieve projects whenever the filters change.
  useEffect(() => {
    let active = true;

    const timer = setTimeout(() => {
      setLoading(true);
      setError(null);

      myProjects({
        name: searchName,
        status: statusFilter || undefined,
      })
        .then((data) => {
          if (active) setProjects(data);
        })
```

7.17 ProjectEditor

frontend/src/pages/ProjectEditor.tsx · line 44

Calls: ApiError, createProject, getCatalogue, getProject, suggestTagsFromUpload, updateProject, uploadImage.

React hooks: useEffect, useNavigate, useParams, useState.

Function	Line
function splitTags(value: string): string[] {	24
function mergeTags(current: string[], extra: string[]): string[] {	31
async function handleSubmit(event: FormEvent<HTMLFormElement>) {	122
async function handleAdd(event: ChangeEvent<HTMLInputElement>) {	193
function removeImage(item: QueueItem) {	241
function takeTag(tag: string) {	247

from line 44 · first 28 lines

```
export default function ProjectEditor() {
  const { projectId } = useParams<{ projectId: string }>();
  const navigate = useNavigate();
  const editing = projectId !== undefined;

  const [title, setTitle] = useState('');
  const [description, setDescription] = useState('');
  const [category, setCategory] = useState('');
  const [style, setStyle] = useState('');
  const [budgetMin, setBudgetMin] = useState('0');
  const [budgetMax, setBudgetMax] = useState('0');
  const [queue, setQueue] = useState<QueueItem[]>([]);
  const [tags, setTags] = useState('');
  const [uploading, setUploading] = useState(false);

  const [catalogue, setCatalogue] = useState<Catalogue | null>(null);
  const [loading, setLoading] = useState(editing);
  const [saving, setSaving] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [success, setSuccess] = useState<string | null>(null);

  // Load the available categories and styles.
  useEffect(() => {
    let active = true;

    getCatalogue()
      .then((data) => {
        if (active) setCatalogue(data);
      })
```

7.18 Profile

frontend/src/pages/Profile.tsx · line 27

Calls: ApiError, createCompany, myCompany, updateMyCompany.

React hooks: useEffect, useState.

Function	Line
function toInput(company: CompanyPublic): CompanyInput {	16
function change(field: keyof CompanyInput) {	62
async function submit(event: FormEvent<HTMLFormElement>) {	69

from line 27 · first 28 lines

```
export default function Profile() {
  const { account } = useAuth();
  const [company, setCompany] = useState<CompanyPublic | null>(null);
  const [form, setForm] = useState<CompanyInput>(EMPTY);
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [notice, setNotice] = useState<string | null>(null);

  useEffect(() => {
    let cancelled = false;

    myCompany()
      .then((found) => {
        if (cancelled) return;
        setCompany(found);
        setForm(toInput(found));
      })
      .catch((caught: unknown) => {
        if (cancelled) return;
        if (caught instanceof ApiError && caught.status === 404) {
          setCompany(null);
          return;
        }
        setError(caught instanceof ApiError ? caught.detail : 'Unable to load the profile.');
      })
      .finally(() => {
        if (!cancelled) setLoading(false);
      });
```

7.19 Admin

frontend/src/pages/Admin.tsx · line 21

Calls: ApiError, adminApprove, adminListAccounts, adminSuspend.

React hooks: useEffect, useState.

Function	Line
function badgeClass(status: Status): string {	17
async function change(accountId: string, action: (id: string) => Promise<AccountPublic>) {	47

from line 21 · first 28 lines

```
export default function Admin() {
  const { account } = useAuth();
  const [rows, setRows] = useState<AccountPublic[]>([]);
  const [filter, setFilter] = useState<Filter>('PENDING');
  const [error, setError] = useState<string | null>(null);
  const [loading, setLoading] = useState(true);
  const [busyId, setBusyId] = useState<string | null>(null);

  const load = useCallback(async () => {
    setLoading(true);
    setError(null);
    try {
      const next = await adminListAccounts(filter === 'ALL' ? undefined : filter);
      setRows(next);
    } catch (caught) {
      setError(caught instanceof ApiError ? caught.detail : 'Unable to load accounts.');
      setRows([]);
    } finally {
      setLoading(false);
    }
  }, [filter]);

  useEffect(() => {
    void load();
  }, [load]);

  async function change(accountId: string, action: (id: string) => Promise<AccountPublic>) {
    setBusyId(accountId);
    setError(null);
```

7.20 Platform

frontend/src/pages/Platform.tsx · line 14

Calls: ApiError, createCategory, getReport, listAllCategories, patchCategory.

React hooks: useEffect, useState.

Function	Line
async function addCategory(event: FormEvent<HTMLFormElement>) {	48
async function toggle(category: CategoryPublic) {	66

from line 14 · first 28 lines

```
export default function Platform() {
  const [categories, setCategories] = useState<CategoryPublic[]>([]);
  const [report, setReport] = useState<Report | null>(null);
  const [period, setPeriod] = useState<(typeof PERIODS)[number]>('daily');
  const [buckets, setBuckets] = useState('14');
  const [newName, setNewName] = useState('');
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [notice, setNotice] = useState<string | null>(null);

  const loadCategories = useCallback(async () => {
    try {
      setCategories(await listAllCategories());
    } catch (caught) {
      setError(caught instanceof ApiError ? caught.detail : 'Unable to load categories.');
    }
  }, []);

  const loadReport = useCallback(async (nextPeriod: string, nextBuckets: string) => {
    try {
      setReport(await getReport(nextPeriod, Number(nextBuckets)));
    } catch (caught) {
      setError(caught instanceof ApiError ? caught.detail : 'Unable to load the report.');
    }
  }, []);

  useEffect(() => {
    void loadCategories();
  }, [loadCategories]);
```

7.21 BudgetSlider

frontend/src/components/BudgetSlider.tsx · line 13

Calls: no endpoint directly; it composes other components.

from line 13 · first 28 lines

```
export default function BudgetSlider({
  min,
  max,
  step,
  low,
  high,
  onChange,
}: BudgetSliderProps) {
  const span = max - min;
  const lowPercent = span === 0 ? 0 : ((low - min) / span) * 100;
  const highPercent = span === 0 ? 100 : ((high - min) / span) * 100;

  return (
    <div className="range">
      <div className="range-track" aria-hidden="true" />
      <div
        className="range-fill"
        aria-hidden="true"
        style={{ left: `${lowPercent}%`, width: `${highPercent - lowPercent}%` }}
      />
      <input
        type="range"
        aria-label="Minimum budget"
        min={min}
        max={max}
        step={step}
        value={low}
        onChange={(event) => {
          const next = Number(event.target.value);
```

7.99 The API client

frontend/src/api.ts · one function per endpoint, all of them typed.

Client function	Line
function getToken()	42
function setToken(token: string)	46
function clearToken()	50
function health()	113
function register(body: RegisterRequest)	117
function login(body: LoginRequest)	124
function me()	131
function adminListAccounts(status?: Status)	135
function adminApprove(accountId: string)	140
function adminSuspend(accountId: string)	144
function myCompany()	148
function createCompany(body: CompanyInput)	152
function updateMyCompany(body: CompanyInput)	159
function companyById(companyId: string)	166
function listCompanies()	170
function searchProjects(params: SearchParams)	174
function getCatalogue()	184
function createProject(body: ProjectInput)	188
function getProject(projectId: string)	197
function updateProject(projectId: string, body: Partial<ProjectInput> & { stat	205
function myProjects(filters: { status?: ProjectStatus; name?: string;	218
function suggestTags(projectId: string, imageIndex = 0)	241
function suggestTagsFromUpload(file: File)	251
function uploadImage(file: File)	257
function recordView(projectId: string)	263
function getShortlist()	269
function saveProject(projectId: string)	273
function unsaveProject(projectId: string)	280
function submitQuoteRequest(body: { company_id: string; title: string; sco	286
function listQuoteRequests(state?: QuoteRequestState)	301
function getQuoteRequest(requestId: string)	306
function counterQuoteRequest(requestId: string, targetCents: number, note: s	312
function cancelQuoteRequest(requestId: string, reason: string)	323
function answerQuoteRequest(requestId: string, body: { total_cents: number; va	333
function reviseQuote(quoteId: string, body: { total_cents: number; valid_days:	343
function refuseQuoteRequest(requestId: string, reason: string)	353
function acceptQuote(quoteId: string)	363
function declineQuote(quoteId: string, reason: string)	369
function listAllCategories()	376
function createCategory(name: string)	380
function patchCategory(categoryId: string, body: { name?: string; active?: boo	387
function getReport(period: string, buckets: number)	397
function deleteProject(projectId: string)	401

8 Sequence walks

Each walk lists the objects in order: boundary, control, entity access, entity. Every step names the file and the line, so a drawing can be traced back to the code. The names below are the names in the source, not summaries.

8.1 A customer searches by photo

#	Role	Object and function	Where	What happens
1	Boundary	Search.tsx · analysePhoto	frontend/src/pages/Search.tsx:111	Reads the chosen file and shows the preview
2	Boundary	api.ts · suggestTagsFromUpload	frontend/src/api.ts:251	POST /api/images/tags with the file
3	Control	images.py · tags_for_upload	backend/app/api/images.py:122	Checks the size and the type, then reads the bytes
4	Control	tag_suggestion_service.py · suggest_tags	backend/app/services/tag_suggestion_service.py:55	Colour palette, object labels, tag list
5	Boundary	Search.tsx · results	frontend/src/pages/Search.tsx	Chips render; searchProjects ranks the catalogue

8.2 A customer requests a quote

#	Role	Object and function	Where	What happens
1	Boundary	RequestQuote.tsx · submit	frontend/src/pages/RequestQuote.tsx:31	Validates the form fields
2	Boundary	api.ts · submitQuoteRequest	frontend/src/api.ts:286	POST /api/quotes/requests
3	Control	quotes.py · submit_request	backend/app/api/quotes.py:63	Rejects a non-customer, maps errors to codes
4	Control	quote_service.py · QuoteService.submit	backend/app/services/quote_service.py:49–88	Builds the request from the payload
5	Entity (access)	quote_repository.py · QuoteRequestRepository.add	backend/app/repositories/quote_repository.py:18–24	Stores the request

8.3 A designer answers a request

#	Role	Object and function	Where	What happens
1	Boundary	QuoteDetail.tsx · submitAnswer	frontend/src/pages/QuoteDetail.tsx:77	Reads total, validity and the GST choice
2	Boundary	api.ts · answerQuoteRequest	frontend/src/api.ts:333	POST /api/quotes/requests/{id}/quotes
3	Control	quotes.py · answer_request	backend/app/api/quotes.py:173	Checks that the caller owns the company
4	Control	quote_service.py · QuoteService.answer	backend/app/services/quote_service.py:138–165	Sends version one when the request is PENDING
5	Entity (access)	quote_repository.py · QuoteRepository.add	backend/app/repositories/quote_repository.py:58–64	Stores the version

8.4 An administrator approves an account

#	Role	Object and function	Where	What happens
1	Boundary	Admin.tsx · change	frontend/src/pages/Admin.tsx:47	Runs the chosen action on one account
2	Boundary	api.ts · adminApprove	frontend/src/api.ts:140	POST /api/admin/accounts/{id}/approve
3	Control	admin.py · approve_account	backend/app/api/admin.py:52	Requires the administrator role
4	Control	account_service.py · AccountService.approve	backend/app/services/account_service.py:86–90	Loads the account and calls the entity
5	Entity	account.py · Account.activate	backend/app/domain/account.py:39–41	Sets the status to ACTIVE

8.5 An administrator reads a report

#	Role	Object and function	Where	What happens
1	Boundary	Platform.tsx · load effect	frontend/src/pages/Platform.tsx:34	Reads period and bucket count
2	Boundary	api.ts · getReport	frontend/src/api.ts:397	GET /api/reports/{period}?buckets={n}
3	Control	reports.py · read_report	backend/app/api/reports.py:46	Requires the administrator role
4	Control	report_service.py · ReportService.build	backend/app/services/report_service.py:57–85	Counts accounts, projects, requests and

				quotes per bucket
5	Entity (access)	repositories · all()	backend/app/repositories	Supplies the rows to count

9 Appendix: every file and what it holds

The index answers the reverse question: given a file, which classes and functions does it contain, and on which lines?

backend/app/api/admin.py _to_http() (22–26) · list_accounts() (34–44) · approve_account() (52–60) · suspend_account() (68–76)

backend/app/api/auth.py register() (40–55) · login() (59–76) · read_current_account() (80–82)

backend/app/api/catalogue.py BudgetRange (17–22) · CatalogueResponse (25–30) · read_catalogue() (38–50)

backend/app/api/categories.py CategoryPublic (25–32) · CategoryInput (35–38) · CategoryPatch (41–45) · _to_public() (48–55) · _fail() (58–65) · list_categories() (73–75) · list_all_categories() (83–91) · create_category() (100–108) · update_category() (116–132)

backend/app/api/companies.py _to_http() (21–31) · create_company() (40–48) · read_my_company() (56–62) · update_my_company() (70–79) · list_companies() (87–89) · read_company() (97–105)

backend/app/api/deps.py get_account_repository() (45–47) · get_account_service() (50–52) · get_company_repository() (55–57) · get_company_service() (60–62) · get_project_repository() (65–67) · get_project_service() (70–72) · get_project_search_service() (75–77) · get_shortlist_repository() (80–82) · get_shortlist_service() (85–87) · get_quote_request_repository() (90–92) · get_quote_repository() (95–97) · get_quote_service() (100–107) · get_category_repository() (110–112) · get_category_service() (115–117) · get_report_service() (120–127) · get_current_account() (130–164)

backend/app/api/health.py HealthResponse (16–21) · read_health() (25–31)

backend/app/api/images.py _read_photo() (58–78) · store_upload() (86–114) · tags_for_upload() (122–169)

backend/app/api/projects.py _to_http() (43–51) · create_project() (60–68) · list_my_projects() (76–95) · list_company_projects() (102–107) · search_projects() (115–153) · read_project() (161–169) · update_project() (177–189) · delete_project() (197–205) · record_project_view() (213–222) · suggest_project_tags() (230–281)

backend/app/api/quotes.py _fail() (37–45) · _detail() (48–54) · submit_request() (63–71) · list_requests() (79–92) · read_request() (100–107) · counter_request() (115–126) · cancel_request() (134–145) · refuse_request() (153–164) · answer_request() (173–184) · list_quotes() (192–200) · revise_quote() (209–220) · accept_quote() (228–236) · decline_quote() (244–255)

backend/app/api/reports.py BucketPublic (22–31) · ReportPublic (34–38) · read_report() (46–83)

backend/app/api/shortlist.py _forbidden() (22–23) · read_shortlist() (31–41) · save_project() (50–67) · unsave_project() (75–83)

backend/app/config.py Settings (30–38) · app_environment() (19–21) · is_production() (24–26) · load_settings() (41–51) · get_secret_key() (54–56) · get_token_minutes() (59–61) · is_default_secret() (64–66) · get_bootstrap_admin_email() (69–71) · get_bootstrap_admin_password() (74–76) · get_bootstrap_admins() (82–100) · _parse_admin_list() (103–120)

backend/app/domain/account.py Account (19–64)

backend/app/domain/category.py Category (22–58) · _now() (17–18)

backend/app/domain/company.py Company (18–88)

backend/app/domain/enums.py Role (8–24) · AccountStatus (27–32) · ProjectStatus (35–39) · QuoteRequestState (42–51) · QuoteState (54–61)

backend/app/domain/errors.py DomainError (12–19) · EmailAlreadyRegistered (22–25) · InvalidCredentials (28–31) · AccountNotActive (34–41) · InvalidRoleForSelfRegistration (53–56) · WeakPassword (59–62) · NotPermitted (65–68) · AccountNotFound (71–74) · CompanyAlreadyExists (77–80) · CompanyNotFound (83–86) · ProjectNotFound (89–92) · QuoteRequestNotFound (95–98) · QuoteNotFound (101–104) · InvalidState (107–110) · inactive_message() (44–50)

backend/app/domain/money.py gst_component() (20–29) · grand_total() (32–36) · format_cents() (39–43)

backend/app/domain/passwords.py hash_password() (19–23) · verify_password() (26–46)

backend/app/domain/project.py SearchCriteria (44–69) · Project (73–313)

backend/app/domain/quote.py Quote (38–131) · _now() (23–24) · _today() (27–34)

backend/app/domain/quote_request.py Counter (28–40) · QuoteRequest (44–163) · _now() (23–24)

backend/app/domain/shortlist.py Shortlist (17–68)

backend/app/main.py _bootstrap_administrators() (43–51) · create_app() (54–91)

backend/app/repositories/account_repository.py AccountRepository (10–59)

backend/app/repositories/category_repository.py CategoryRepository (11–62)

backend/app/repositories/company_repository.py CompanyRepository (10–57)

backend/app/repositories/project_repository.py ProjectRepository (10–57)

backend/app/repositories/quote_repository.py QuoteRequestRepository (11–48) · QuoteRepository (51–92)

backend/app/repositories/shortlist_repository.py ShortlistRepository (10–35)

backend/app/schemas/account.py AccountPublic (12–22) · RegisterRequest (25–48) · LoginRequest (51–55) · TokenResponse (58–64)

backend/app/schemas/company.py CompanyPublic (10–24) · CompanyCreate (27–35) · CompanyUpdate (38–50)

backend/app/schemas/project.py ProjectPublic (12–32) · ProjectCreate (35–45) · ProjectUpdate (48–62) · SearchResponse (65–72) · PaletteEntryPublic (75–79) · TagSuggestionPublic (82–91) · StoredImagePublic (94–99)

backend/app/schemas/quote.py CounterPublic (12–20) · QuotePublic (23–39) · QuoteRequestPublic (42–60) · QuoteRequestDetail (63–66) · SubmitRequest (69–78) · CounterRequest (81–85) · AnswerRequest (88–94) · ReasonRequest (97–100)

backend/app/schemas/shortlist.py ShortlistResponse (10–14)

backend/app/security.py create_access_token() (19–28) · decode_access_token() (31–44)

backend/app/services/account_service.py AccountService (31–141)

backend/app/services/category_service.py CategoryNotFound (12–16) · CategoryExists (19–23) · CategoryService (26–87)

backend/app/services/company_service.py CompanyService (12–100)

backend/app/services/image_source.py ImageUnavailable (28–29) · load_image_bytes() (32–39) · _read_local() (42–51) · _read_remote() (54–82) · _resolve() (85–90) · _is_public() (93–105)

backend/app/services/object_labels.py Reading (162–169) · is_animal() (122–129) · _session() (133–153) · available() (156–158) · read() (172–231) · object_tags() (234–241) · engine_name() (244–246)

backend/app/services/project_search_service.py SearchResult (20–33) · ProjectSearchService (36–92)

backend/app/services/project_service.py ProjectService (13–178)

backend/app/services/quote_service.py QuoteService (32–312)

backend/app/services/report_service.py Bucket (22–31) · Report (35–39) · ReportService (42–133)

backend/app/services/shortlist_service.py ShortlistService (14–72)

backend/app/services/tag_suggestion_service.py PaletteEntry (36–40) · TagSuggestion (44–52) · suggest_tags() (55–82) · _merge() (85–91) · _palette() (94–109) · _stats() (112–124) · _tags() (127–150)